

SOLIDWORKS API (.NET VBA C#) — Examples (9/50):

Create_Base_Flange_Feature_Example_VBNET-

Create_Circular_Pattern_of_Subassembly_E

SOLIDWORKS API Documentation (converted from CHM)

Create Base Flange Feature Example (VB.NET)

This example shows how to create a base flange feature.

```
'=====
'Preconditions:
'1. Ensure the specified part template exists.
'2. Open the Immediate window.
'
'Postconditions:
'1. Creates a flange profile sketch.
'2. Creates Base-Flange1 in the FeatureManager design tree.
'3. Inspect the Immediate window, graphics area,
'   and FeatureManager design tree.
'=====
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
```

```
    Dim swPart As PartDoc
```

```
    Dim swModel As ModelDoc2
```

```
    Dim swSKFeat As Feature
```

```
    Dim skSegment As SketchSegment
```

```
    Dim swBaseFlangeFeat As BaseFlangeFeatureData
```

```
    Dim baseFlangeFeatData As BaseFlangeFeatureData
```

```
    Dim cba As CustomBendAllowance
```

```
    Dim var() As Object
```

```
    Dim parent As Feature
```

```
    Dim SHFeat As Feature
```

```
    Dim smFeatData As SheetMetalFeatureData
```

```
    Dim cba1 As CustomBendAllowance
```

```
    Dim boolstatus As Boolean
```

```
    Dim longstatus As Long, longwarnings As Long
```

```
Sub main()
```

```
    Part = swApp.ActiveDoc
```

```
    Dim swSheetWidth As Double
```

```
    swSheetWidth = 0
```

```
    Dim swSheetHeight As Double
```

```
    swSheetHeight = 0
```

```
    Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Part.prtdot", 0, swSheetWidth, swSheetHeight)
```

```
    swPart = Part
```

```
    swModel = swApp.ActiveDoc
```

```
    boolstatus = Part.Extension.SelectByID2("Top", "PLANE", -0.0598881514598713, 0.0393749830258702, 0.00485137895479469,
False, 0, Nothing, 0)
```

```
    Part.SketchManager.InsertSketch(True)
```

```
    Part.ClearSelection2(True)
```

```
    skSegment = Part.SketchManager.CreateLine(-0.140779, 0.050824, 0#, -0.106481, -0.06735, 0#)
```

```
    skSegment = Part.SketchManager.CreateLine(-0.106481, -0.06735, 0#, 0.084966, -0.049265, 0#)
```

```

skSegment = Part.SketchManager.CreateLine(0.084966, -0.049265, 0#, 0.143274, 0.063608, 0#)
Part.ClearSelection2(True)
Part.SketchManager.InsertSketch(True)

swSKFeat = swModel.SelectionManager.GetSelectedObject6(1, -1)
Debug.Print("Flange profile name : " & swSKFeat.Name & " and type : " & swSKFeat.GetTypeName2)

swBaseFlangeFeat = swModel.FeatureManager.CreateDefinition(swFeatureNameID_e.swFmBaseFlange)

cba = swBaseFlangeFeat.GetCustomBendAllowance()

cba.Type = swBendAllowanceTypes_e.swBendAllowanceDirect
cba.BendAllowance = 0.05

swBaseFlangeFeat.D1EndConditionType = 1
swBaseFlangeFeat.D1EndConditionDistance = 0.02
swBaseFlangeFeat.ReverseDirection = True
swBaseFlangeFeat.OffsetDirections = 2
swBaseFlangeFeat.D2EndConditionType = 1
swBaseFlangeFeat.D2EndConditionDistance = 0.05
swBaseFlangeFeat.OverrideDefaultSheetMetalParameters = True
swBaseFlangeFeat.Thickness = 0.035

'Initialize the base flange feature
'Initialize(
'UseMaterialSheetMetalParameters=False,
'UseDefaultBendAllowance=False,
'CustomBendAllowance,
'UseDefaultBendRelief=False,
'ReliefType=swSheetMetalReliefTypes_e.swSheetMetalReliefRectangular,
'UseReliefRatio=True,
'ReliefRatio=0.8m,
'ReliefWidth,
'ReliefDepth)

0.8, 0#, 0#)
Call swBaseFlangeFeat.Initialize(False, False, cba, False, swSheetMetalReliefTypes_e.swSheetMetalReliefRectangular, True,

SHFeat = swModel.FeatureManager.CreateFeature(swBaseFlangeFeat)

baseFlangeFeatData = SHFeat.GetDefinition()

Debug.Print("Use material sheet metal parameters? " & baseFlangeFeatData.UseMaterialSheetMetalParameters)
Debug.Print("Use default bend allowance? " & baseFlangeFeatData.UseDefaultBendAllowance)
Debug.Print("Use default bend relief? " & baseFlangeFeatData.UseDefaultBendRelief)
Debug.Print("Use relief ratio? " & baseFlangeFeatData.UseReliefRatio)
Debug.Print("Relief type as defined by swSheetMetalReliefTypes_e: " & baseFlangeFeatData.ReliefType)
Debug.Print("Relief width: " & baseFlangeFeatData.ReliefWidth)
Debug.Print("Relief depth: " & baseFlangeFeatData.ReliefDepth)
Debug.Print("Relief ratio: " & baseFlangeFeatData.ReliefRatio)

'Modify the relief ratio and override default AutoRelief in the parent sheet metal feature
var = SHFeat.GetParents()
parent = var(1)
Debug.Print("Parent type: " & parent.GetTypeName2())

smFeatData = parent.GetDefinition()

cba1 = smFeatData.GetCustomBendAllowance()

Debug.Print("Custom bend allowance type as defined in swBendAllowanceTypes_e: " & cba1.Type)
Debug.Print("Bend allowance: " & cba1.BendAllowance)

```

```

        Debug.Print("Result code for override of AutoRelief as defined by swSheetMetalModifierError_e: " &
smFeatData.SetOverrideDefaultParameter2(swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_AutoRelief,
True))
        smFeatData.ReliefRatio = 0.7

        Debug.Print("Base flange successfully modified? " & parent.ModifyDefinition(smFeatData, swModel, Nothing))
        Debug.Print("Base flange feature name : " & SHFeat.Name & " and type : " & SHFeat.GetTypeName2)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Create Belt/Chain Feature Example (C#)

This example shows how to create a Belt/Chain assembly feature.

```
// =====
// Preconditions: Ensure the specified assembly template and part exist.
//
// Postconditions:
// 1. Creates an assembly of two steel discs.
// 2. Press F5 to create Belt1 in the FeatureManager design tree.
//    If a Save As dialog appears, click Cancel.
// 3. Modifies Belt1's belt length, belt thickness,
//    and pulley diameters. If a Save As dialog appears, click Cancel.
// =====

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Forms;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace Macro1_CSharp
{
    public partial class SolidWorksMacro
    {
        public Dispatcher[] ObjectArrayToDispatcherArray(object[] Objects)
        {
            int ArraySize;
            ArraySize = Objects.GetUpperBound(0);
            var d = new Dispatcher[ArraySize + 1];
            int ArrayIndex;
            var loopTo = ArraySize;
            for (ArrayIndex = 0; ArrayIndex <= loopTo; ArrayIndex++)
                d[ArrayIndex] = new Dispatcher(Objects[ArrayIndex]);
            return d;
        }

        public void Main()
        {
            ModelDoc2 swModel;
            SelectionMgr selMgr;
            BeltChainFeatureData beltChainFeatData;
            var pulleyComps = new object[2];
            var pulleyDiams = new double[2];
            var pulleyFlips = new bool[2];
            var pulleyDiamsOri = new double[2];
            Feature swFeat;
            BeltChainFeatureData swFeatData;
            int I;
            int J;
            bool boolstatus;
            var longwarnings = default(int);
            double swSheetWidth;
            swSheetWidth = 0d;
            double swSheetHeight;
```

```

        swSheetHeight = 0d;
        swModel = (ModelDoc2)swApp.NewDocument(@"C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Assembly.asmdot", 0,
swSheetWidth, swSheetHeight);
        AssemblyDoc swAssembly;
        swAssembly = (AssemblyDoc)swModel;

        // Insert steel disc component

        string AssemblyTitle;
        AssemblyTitle = swModel.GetTitle();
        ModelDoc2 tmpObj;
        var errors = default(int);
        tmpObj = swApp.OpenDoc6(@"C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2022\samples\tutorial\api\AdvancedMates\steel
disc.sldprt", 1, 1, "", errors, longwarnings);
        swModel = (ModelDoc2)swApp.ActivateDoc3(AssemblyTitle, true, 0, ref errors);
        Component2 swInsertedComponent;
        swInsertedComponent = ((AssemblyDoc)swModel).AddComponent5(@"C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2022\samples\tutorial\api\AdvancedMates\steel disc.sldprt", 0, "", false, "", 0.00404421078452853d, 0.00404421078422676d,
0.0122628871084471d);
        swApp.CloseDoc(@"C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2022\samples\tutorial\api\AdvancedMates\steel
disc.sldprt");

        // Insert steel disc component

        AssemblyTitle = swModel.GetTitle();
        tmpObj = swApp.OpenDoc6(@"C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2022\samples\tutorial\api\AdvancedMates\steel
disc.sldprt", 1, 1, "", errors, longwarnings);
        swModel = (ModelDoc2)swApp.ActivateDoc3(AssemblyTitle, true, 0, errors);
        swInsertedComponent = ((AssemblyDoc)swModel).AddComponent5(@"C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2022\samples\tutorial\api\AdvancedMates\steel disc.sldprt", 0, "", false, "", 0.0200044210784528d, 0.0200044210784227d,
0.0122628871084471d);
        swApp.CloseDoc(@"C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2022\samples\tutorial\api\AdvancedMates\steel
disc.sldprt");

        // Float the components

        boolstatus = swModel.Extension.SelectByID2("steel disc-1", "COMPONENT", 0, 0, 0, false, 0, null, 0);
        //swModel.UnfixComponent();
        swModel.ClearSelection2(true);
        boolstatus = swModel.Extension.SelectByID2("steel disc-2", "COMPONENT", 0, 0, 0, false, 0, null, 0);
        //swModel.UnfixComponent();
        swModel.ClearSelection2(true);

        // Move one disc

        boolstatus = swModel.Extension.SelectByID2("steel disc-2@Assem14", "COMPONENT", 0, 0, 0, false, 0, null, 0);
        boolstatus = swModel.Extension.SelectByID2("Unknown", "MANIPULATOR", 0.0340009836225566d, -0.00352899570186584d,
-0.0115173288530189d, false, 0, null, 0);
        swModel.ClearSelection2(true);
        boolstatus = swModel.Extension.SelectByID2("steel disc-2@Assem14", "COMPONENT", 0, 0, 0, false, 0, null, 0);
        var TransformData = new double[16];
        TransformData[0] = 1d;
        TransformData[1] = 0d;
        TransformData[2] = 0d;
        TransformData[3] = 0d;
        TransformData[4] = 1d;
        TransformData[5] = 0d;
        TransformData[6] = 0d;
        TransformData[7] = 0d;
        TransformData[8] = 1d;
        TransformData[9] = 0.0664528131333411d;
        TransformData[10] = 0.0046349356621274d;
        TransformData[11] = 0.00919716533133533d;
        TransformData[12] = 1d;
        TransformData[13] = 0d;
        TransformData[14] = 0d;

```

```

TransformData[15] = 0d;
object TransformDataVariant;
TransformDataVariant = TransformData;
MathUtility swMathUtil;
swMathUtil = (MathUtility)swApp.GetMathUtility();
MathTransform swTransform;
swTransform = (MathTransform)swMathUtil.CreateTransform(TransformDataVariant);
Component2 swComp;
swComp = (Component2)((SelectionMgr)swModel.SelectionManager).GetSelectedObjectsComponent4(1, -1);
boolstatus = swComp.SetTransformAndSolve2(swTransform);
boolstatus = swModel.ForceRebuild3(false);
swModel.ClearSelection2(true);

// Select the pulley faces

boolstatus = swModel.Extension.SelectByRay(0.0130157615084272d, 0.00990176398499898d, 0.0121640119419908d,
-0.333333333266778d, -0.333333333254022d, -0.881917103743329d, 0.000973554376657825d, 2, false, 0, 0);
boolstatus = swModel.Extension.SelectByRay(0.0769710902559098d, 0.00667589089525222d, 0.0112969076145077d,
-0.333333333266778d, -0.333333333254022d, -0.881917103743329d, 0.000973554376657825d, 2, false, 0, 0);
selMgr = (SelectionMgr)swModel.SelectionManager;
var loopTo = pulleyComps.GetUpperBound(0);
for (I = 0; I <= loopTo; I++)
    pulleyComps[I] = selMgr.GetSelectedObject6(I + 1, -1);

// Create Belt/Chain feature

beltChainFeatData =
(BeltChainFeatureData)swModel.FeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmBeltAndChain);
var loopTo1 = pulleyComps.GetUpperBound(0);
for (J = 0; J <= loopTo1; J++)
    pulleyDiams[J] = 0.005d * (J + 2);

DispatchWrapper[] dArray;
dArray = ObjectArrayToDispatchWrapperArray(pulleyComps);
beltChainFeatData.PulleyComponents = dArray;
beltChainFeatData.PulleyDiameters = pulleyDiams;
pullyFlips[0] = false;
pullyFlips[1] = false;
beltChainFeatData.FlipSides = pullyFlips;
beltChainFeatData.BeltLocationPlane = selMgr.GetSelectedObject6(3, -1);
beltChainFeatData.DrivingLength = true;
beltChainFeatData.BeltLength = 0.59492d;
beltChainFeatData.UseBeltThickness = true;
beltChainFeatData.BeltThickness = 0.003d;
beltChainFeatData.CreateBeltPart = true;
beltChainFeatData.EngageBelt = false;
Debugger.Break();
swFeat = swModel.FeatureManager.CreateFeature(beltChainFeatData);
swFeatData = (BeltChainFeatureData)swFeat.GetDefinition();

// Modify Belt/Chain feature

boolstatus = swFeatData.AccessSelections(swModel, null);
pulleyDiamsOri = (double[])swFeatData.PulleyDiameters;
pulleyDiamsOri[0] = 0.02d;
pulleyDiamsOri[1] = 0.02d;
swFeatData.PulleyDiameters = pulleyDiamsOri;
Debug.Print("Pulley diameters changed to 20 mm");
swFeatData.DrivingLength = true;
swFeatData.BeltLength = 0.3d;
Debug.Print("Belt length (m) changed to " + swFeatData.BeltLength);
swFeatData.BeltThickness = 0.002d;
Debug.Print("Belt thickness (m) changed to " + swFeatData.BeltThickness);
boolstatus = swFeat.ModifyDefinition(swFeatData, swModel, null);

```

```
}
```

```
// The SldWorks swApp variable is pre-assigned for you.
```

```
public SldWorks swApp;
```

```
}
```

```
}
```

Create Belt/Chain Feature Example (VB.NET)

This example shows how to create a Belt/Chain assembly feature.

```
'=====
'Preconditions: Ensure the specified assembly template and part exist.
'
'Postconditions:
'1. Creates an assembly of two steel discs.
'2. Press F5 to create Belt1 in the FeatureManager design tree.
'   If a Save As dialog appears, click Cancel.
'3. Modifies Belt1's belt length, belt thickness,
'   and pulley diameters. If a Save As dialog appears, click Cancel.
'=====
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
Function ObjectArrayToDispatcherArray(ByVal Objects As Object()) As DispatcherArray()
```

```
    Dim ArraySize As Integer
```

```
    ArraySize = Objects.GetUpperBound(0)
```

```
    Dim d(ArraySize) As Dispatcher
```

```
    Dim ArrayIndex As Integer
```

```
    For ArrayIndex = 0 To ArraySize
```

```
        d(ArrayIndex) = New Dispatcher(Objects(ArrayIndex))
```

```
    Next
```

```
    Return d
```

```
End Function
```

```
Sub main()
```

```
    Dim swModel As ModelDoc2
```

```
    Dim selMgr As SelectionMgr
```

```
    Dim beltChainFeatData As BeltChainFeatureData
```

```
    Dim pulleyComps(1) As Object
```

```
    Dim pulleyDiams(1) As Double
```

```
    Dim pulleyFlips(1) As Boolean
```

```
    Dim pulleyDiamsOri(1) As Double
```

```
    Dim swFeat As Feature
```

```
    Dim swFeatData As BeltChainFeatureData
```

```
    Dim I As Integer
```

```
    Dim J As Integer
```

```
    Dim boolstatus As Boolean
```

```
    Dim longwarnings As Integer
```

```
    Dim swSheetWidth As Double
```

```
    swSheetWidth = 0
```

```
    Dim swSheetHeight As Double
```

```
    swSheetHeight = 0
```

```
    swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Assembly.asmdot", 0, swSheetWidth, swSheetHeight)
```

```
    Dim swAssembly As AssemblyDoc
```

```
    swAssembly = swModel
```

```
    ' Insert steel disc component
```

```

Dim AssemblyTitle As String
AssemblyTitle = swModel.GetTitle
Dim tmpObj As ModelDoc2
Dim errors As Long
tmpObj = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2022\samples\tutorial\api\AdvancedMates\steel
disc.sldprt", 1, 1, "", errors, longwarnings)
swModel = swApp.ActivateDoc3(AssemblyTitle, True, 0, errors)
Dim swInsertedComponent As Component2
swInsertedComponent = swModel.AddComponent5("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2022\samples\tutorial\api\AdvancedMates\steel disc.sldprt", 0, "", False, "", 0.00404421078452853, 0.00404421078422676,
0.0122628871084471)
swApp.CloseDoc("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2022\samples\tutorial\api\AdvancedMates\steel disc.sldprt")

' Insert steel disc component

AssemblyTitle = swModel.GetTitle
tmpObj = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2022\samples\tutorial\api\AdvancedMates\steel
disc.sldprt", 1, 1, "", errors, longwarnings)
swModel = swApp.ActivateDoc3(AssemblyTitle, True, 0, errors)
swInsertedComponent = swModel.AddComponent5("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2022\samples\tutorial\api\AdvancedMates\steel disc.sldprt", 0, "", False, "", 0.0200044210784528, 0.0200044210784227,
0.0122628871084471)
swApp.CloseDoc("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2022\samples\tutorial\api\AdvancedMates\steel disc.sldprt")

' Float the components

boolstatus = swModel.Extension.SelectByID2("steel disc-1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
swModel.UnfixComponent
swModel.ClearSelection2(True)
boolstatus = swModel.Extension.SelectByID2("steel disc-2", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
swModel.UnfixComponent
swModel.ClearSelection2(True)

' Move one disc

boolstatus = swModel.Extension.SelectByID2("steel disc-2@Assem14", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
boolstatus = swModel.Extension.SelectByID2("Unknown", "MANIPULATOR", 0.0340009836225566, -0.00352899570186584,
-0.0115173288530189, False, 0, Nothing, 0)
swModel.ClearSelection2(True)
boolstatus = swModel.Extension.SelectByID2("steel disc-2@Assem14", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
Dim TransformData(15) As Double
TransformData(0) = 1
TransformData(1) = 0
TransformData(2) = 0
TransformData(3) = 0
TransformData(4) = 1
TransformData(5) = 0
TransformData(6) = 0
TransformData(7) = 0
TransformData(8) = 1
TransformData(9) = 0.0664528131333411
TransformData(10) = 0.0046349356621274
TransformData(11) = 0.00919716533133533
TransformData(12) = 1
TransformData(13) = 0
TransformData(14) = 0
TransformData(15) = 0
Dim TransformDataVariant As Object
TransformDataVariant = TransformData
Dim swMathUtil As Object
swMathUtil = swApp.GetMathUtility()
Dim swTransform As Object
swTransform = swMathUtil.CreateTransform((TransformDataVariant))
Dim swComp As Object

```

```

swComp = swModel.SelectionManager.GetSelectedObjectsComponent4(1, -1)
boolstatus = swComp.SetTransformAndSolve2(swTransform)
boolstatus = swModel.ForceRebuild3(False)
swModel.ClearSelection2(True)

' Select the pulley faces

boolstatus = swModel.Extension.SelectByRay(0.0130157615084272, 0.00990176398499898, 0.0121640119419908, -0.33333333266778,
-0.33333333254022, -0.881917103743329, 0.000973554376657825, 2, False, 0, 0)
boolstatus = swModel.Extension.SelectByRay(0.0769710902559098, 0.00667589089525222, 0.0112969076145077, -0.33333333266778,
-0.33333333254022, -0.881917103743329, 0.000973554376657825, 2, False, 0, 0)

selMgr = swModel.SelectionManager

For I = 0 To UBound(pulleyComps)
    pulleyComps(I) = selMgr.GetSelectedObject6(I + 1, -1)
Next

' Create Belt/Chain feature

beltChainFeatData = swModel.FeatureManager.CreateDefinition(swFeatureNameID_e.swFmBeltAndChain)

For J = 0 To UBound(pulleyComps)
    pulleyDiams(J) = 0.005 * (J + 2)
Next

Dim dArray() As DispatcherWrapper

dArray = ObjectArrayToDispatcherWrapperArray(pulleyComps)

beltChainFeatData.PulleyComponents = dArray
beltChainFeatData.PulleyDiameters = pulleyDiams

pullyFlips(0) = False
pullyFlips(1) = False

beltChainFeatData.FlipSides = pullyFlips

beltChainFeatData.BeltLocationPlane = selMgr.GetSelectedObject6(3, -1)

beltChainFeatData.DrivingLength = True
beltChainFeatData.BeltLength = 0.59492

beltChainFeatData.UseBeltThickness = True
beltChainFeatData.BeltThickness = 0.003

beltChainFeatData.CreateBeltPart = True
beltChainFeatData.EngageBelt = False

Stop

swFeat = (swModel.FeatureManager).CreateFeature(beltChainFeatData)

swFeatData = swFeat.GetDefinition()

' Modify Belt/Chain feature

boolstatus = swFeatData.AccessSelections(swModel, Nothing)

pulleyDiamsOri = swFeatData.PulleyDiameters
pulleyDiamsOri(0) = 0.02
pulleyDiamsOri(1) = 0.02

```

```
swFeatData.PulleyDiameters = pulleyDiamsOri
Debug.Print("Pulley diameters changed to 20 mm")

swFeatData.DrivingLength = True
swFeatData.BeltLength = 0.3
Debug.Print("Belt length (m) changed to " & swFeatData.BeltLength)

swFeatData.BeltThickness = 0.002
Debug.Print("Belt thickness (m) changed to " & swFeatData.BeltThickness)

boolstatus = swFeat.ModifyDefinition(swFeatData, swModel, Nothing)
```

End Sub

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
```

```
Public swApp As SldWorks
```

End Class

Create Bidirectional Linear Pattern Example (C#)

This example shows how to create a bidirectional linear pattern feature.

```
//-----
// Preconditions: Verify that the part exists.
//
// Postconditions:
// 1. Opens the part.
// 2. Selects the feature to pattern.
// 3. Selects the edges for Direction 1 and Direction 2
//    of the bidirectional linear pattern.
// 4. Creates LPattern1.
// 5. Examine the FeatureManager design tree and graphics area.
//
// NOTE: Because the part is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateLinearPattern_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private ModelDocExtension swModelDocExt;
        private FeatureManager swFeatureManager;
        private Feature swFeature;
        private LinearPatternFeatureData swLinearPatternFeatureData;
        private bool status;
        private int errors = 0;
        private int warnings = 0;
        private string fileName;
        public void Main()
        {
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\box.sldprt";
            swModel = swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "");
            swModelDocExt = swModel.Extension;
            swFeatureManager = swModel.FeatureManager;
            //Select feature to pattern
            status = swModelDocExt.SelectByID2("CBORE for #6 Binding Head Machine Screw1", "BODYFEATURE", 0, 0, 0, false, 4, null, 0);

            //Select edges for Direction 1 and Direction 2
            status = swModelDocExt.SelectByID2("", "EDGE", -0.0341223962029176, 0.0300321434688158, 0.0460303188361877, true, 1, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.0436465176948104, 0.0301916139486593, 0.0114344277122314, true, 2, null, 0);

            //Create linear pattern
            swLinearPatternFeatureData = (LinearPatternFeatureData)swFeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmLPat
            swLinearPatternFeatureData.D1EndCondition = 0;
            swLinearPatternFeatureData.D1ReverseDirection = false;
            swLinearPatternFeatureData.D1Spacing = 0.01;
            swLinearPatternFeatureData.D1TotalInstances = 4;
            swLinearPatternFeatureData.D2EndCondition = 0;
            swLinearPatternFeatureData.D2PatternSeedOnly = false;
            swLinearPatternFeatureData.D2ReverseDirection = false;
            swLinearPatternFeatureData.D2Spacing = 0.01;
            swLinearPatternFeatureData.D2TotalInstances = 4;
            swLinearPatternFeatureData.GeometryPattern = false;
            swLinearPatternFeatureData.VarySketch = false;
            swFeature = swFeatureManager.CreateFeature(swLinearPatternFeatureData);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Bidirectional Circular Pattern Feature Example (C#)

This example shows how to create a bidirectional circular pattern feature.

```
//-----
// Preconditions:
// 1. Verify that the part exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part.
// 2. Selects a feature.
// 3. Selects an edge for Direction 1.
// 4. Creates a bidirectional circular pattern.
// 5. Examine the FeatureManager design tree,
//    graphics area, and Immediate window.
//
// NOTE: Because the part is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            CircularPatternFeatureData swCircularPatternFeatureData = default(CircularPatternFeatureData);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2019\\samples\\tutorial\\api\\featurecircularpattern.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            status = swModelDocExt.SelectByID("Boss-
Extrude2", "BODYFEATURE", -0.0000568552547690615, 0.0336059294503599, 0.06999999999999932, false, 4, null, 0);
            status = swModelDocExt.SelectByRay(0.0289184346104037, 0.0205122863998071, 0.0598787397922251, 0.342497149434059, -0.155
            swCircularPatternFeatureData =
            (CircularPatternFeatureData)swFeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmCirPattern);
            swFeature = (Feature)swFeatureManager.CreateFeature(swCircularPatternFeatureData);
            swCircularPatternFeatureData = (CircularPatternFeatureData)swFeature.GetDefinition();
            Debug.Print("Direction 1:");
            Debug.Print(" Equal spacing: " + swCircularPatternFeatureData.EqualSpacing);
            Debug.Print(" Spacing: " + swCircularPatternFeatureData.Spacing);
            Debug.Print(" Total instances: " + swCircularPatternFeatureData.TotalInstances);
            if (swCircularPatternFeatureData.Direction2 == true)
            {
                Debug.Print("Direction 2:");
                Debug.Print(" Symmetric: " + swCircularPatternFeatureData.Symmetric);
                Debug.Print(" Equal spacing: " + swCircularPatternFeatureData.EqualSpacing2);
                Debug.Print(" Spacing: " + swCircularPatternFeatureData.Spacing2);
                Debug.Print(" Total instances: " + swCircularPatternFeatureData.TotalInstances2);
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Create Bidirectional Circular Pattern Feature Example (VBA)

This example shows how to create a bidirectional circular pattern feature.

```

'-----
' Preconditions:
' 1. Verify that the part exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Selects a feature.
' 3. Selects an edge for Direction 1.
' 4. Creates a bidirectional circular pattern.
' 5. Examine the FeatureManager design tree,
'    graphics area, and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swCircularPatternFeatureData As SldWorks.CircularPatternFeatureData
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\featurecircularpattern.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,
warnings)
    Set swModelDocExt = swModel.Extension
    Set swFeatureManager = swModel.FeatureManager
    status = swModelDocExt.SelectByID2("Boss-Extrude2", "BODYFEATURE", -5.68552547690615E-05, 3.36059294503599E-02,
6.9999999999932E-02, False, 4, Nothing, 0)
    status = swModelDocExt.SelectByRay(2.89184346104037E-02, 2.05122863998071E-02, 5.98787397922251E-02, 0.342497149434059,
-0.159204982974168, -0.925931679998983, 9.12809005339891E-04, 1, True, 1, 0)

    Set swCircularPatternFeatureData = swFeatureManager.CreateDefinition(swFmCirPattern)
    Set swFeature = swFeatureManager.CreateFeature(swCircularPatternFeatureData)

    Set swCircularPatternFeatureData = swFeature.GetDefinition
    Debug.Print "Direction 1:"
    Debug.Print " Equal spacing: " & swCircularPatternFeatureData.EqualSpacing
    Debug.Print " Spacing: " & swCircularPatternFeatureData.Spacing
    Debug.Print " Total instances: " & swCircularPatternFeatureData.TotalInstances
    If swCircularPatternFeatureData.Direction2 Then
        Debug.Print "Direction 2:"
        Debug.Print " Symmetric: " & swCircularPatternFeatureData.Symmetric
        Debug.Print " Equal spacing: " & swCircularPatternFeatureData.EqualSpacing2
        Debug.Print " Spacing: " & swCircularPatternFeatureData.Spacing2
        Debug.Print " Total instances: " & swCircularPatternFeatureData.TotalInstances2
    End If
End Sub

```

Create Bidirectional Circular Pattern Feature Example (VB.NET)

This example shows how to create a bidirectional circular pattern feature.

```

'-----
' Preconditions:
' 1. Verify that the part exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Selects a feature.
' 3. Selects an edge for Direction 1.
' 4. Creates a bidirectional circular pattern.
' 5. Examine the FeatureManager design tree,
'    graphics area, and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim swCircularPatternFeatureData As CircularPatternFeatureData
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\featurecircularpattern.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warr
        swModelDocExt = swModel.Extension
        swFeatureManager = swModel.FeatureManager
        status = swModelDocExt.SelectByID2("Boss-
Extrude2", "BODYFEATURE", -0.0000568552547690615, 0.0336059294503599, 0.06999999999999932, False, 4, Nothing, 0)
        status = swModelDocExt.SelectByRay(0.0289184346104037, 0.0205122863998071, 0.0598787397922251, 0.342497149434059, -0.1592045
        swCircularPatternFeatureData = swFeatureManager.CreateDefinition(swFeatureNameID_e.swFmCirPattern)
        swFeature = swFeatureManager.CreateFeature(swCircularPatternFeatureData)

        swCircularPatternFeatureData = swFeature.GetDefinition
        Debug.Print("Direction 1:")
        Debug.Print(" Equal spacing: " & swCircularPatternFeatureData.EqualSpacing)
        Debug.Print(" Spacing: " & swCircularPatternFeatureData.Spacing)
        Debug.Print(" Total instances: " & swCircularPatternFeatureData.TotalInstances)
        If swCircularPatternFeatureData.Direction2 Then
            Debug.Print("Direction 2:")
            Debug.Print(" Symmetric: " & swCircularPatternFeatureData.Symmetric)
            Debug.Print(" Equal spacing: " & swCircularPatternFeatureData.EqualSpacing2)
            Debug.Print(" Spacing: " & swCircularPatternFeatureData.Spacing2)
            Debug.Print(" Total instances: " & swCircularPatternFeatureData.TotalInstances2)
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Create Bidirectional Linear Pattern Example (C#)

This example shows how to create a bidirectional linear pattern.

```
//-----
// Preconditions: Verify that the part exists.
//
// Postconditions:
// 1. Opens the part.
// 2. Selects the feature to pattern.
// 3. Selects the edges for Direction 1 and
// Direction 2 for the bidirectional linear
// pattern.
// 4. Creates a bidirectional linear pattern.
// 5. Examine the FeatureManager design tree and
// graphics area.
//
// NOTE: Because the part is used elsewhere, do not save
// changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\box.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;

            //Select feature to pattern
            status = swModelDocExt.SelectByID2("CBORE for #6 Binding Head Machine Screw1", "BODYFEATURE", 0, 0, 0, false, 4, null, 0);

            //Select edges for Direction 1 and Direction 2
            status = swModelDocExt.SelectByID2("", "EDGE", -0.0341223962029176, 0.0300321434688158, 0.0460303188361877, true, 1, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.0436465176948104, 0.0301916139486593, 0.0114344277122314, true, 2, null, 0);

            //Create linear pattern
            swFeature = (Feature)swFeatureManager.FeatureLinearPattern5(3, 0.01, 3, 0.01, false, false, "NULL", "NULL", true, false,
false, false, false, false, true, true, false, false, 0, 0, false,
false);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Bidirectional Linear Pattern Example (VBA)

This example shows how to create a bidirectional linear pattern.

```
'-----
' Preconditions: Verify that the part exists.
'
' Postconditions:
' 1. Opens the part.
' 2. Selects the feature to pattern.
' 3. Selects the edges for Direction 1 and
'    Direction 2 for the bidirectional linear
'    pattern.
' 4. Creates a bidirectional linear pattern.
' 5. Examine the FeatureManager design tree and
'    graphics area.
'
' NOTE: Because the part is used elsewhere, do not save
' changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swFeatureManager = swModel.FeatureManager

    'Select feature to pattern
    status = swModelDocExt.SelectByID2("CB0RE for #6 Binding Head Machine Screw1", "BODYFEATURE", 0, 0, 0, False, 4, Nothing, 0)

    'Select edges for Direction 1 and Direction 2
    status = swModelDocExt.SelectByID2("", "EDGE", -3.41223962029176E-02, 3.00321434688158E-02, 4.60303188361877E-02, True, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "EDGE", 4.36465176948104E-02, 3.01916139486593E-02, 1.14344277122314E-02, True, 2, Nothing, 0)

    'Create linear pattern
    Set swFeature = swFeatureManager.FeatureLinearPattern5(3, 0.01, 3, 0.01, False, False, "NULL", "NULL", True, False, False, False, False, True,
    True, False, False, 0, 0, False, False)

End Sub
```

Create Bidirectional Linear Pattern Example (VB.NET)

This example shows how to create a bidirectional linear pattern.

```

-----
Preconditions: Verify that the part exists.

Postconditions:
1. Opens the part.
2. Selects the feature to pattern.
3. Selects the edges for Direction 1 and
   Direction 2 for the bidirectional linear
   pattern.
4. Creates a bidirectional linear pattern.
5. Examine the FeatureManager design tree and
   graphics area.

NOTE: Because the part is used elsewhere, do not save
changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swFeatureManager = swModel.FeatureManager

        'Select feature to pattern
        status = swModelDocExt.SelectByID2("CBORE for #6 Binding Head Machine Screw1", "BODYFEATURE", 0, 0, 0, False, 4, Nothing, 0)

        'Select edges for Direction 1 and Direction 2
        status = swModelDocExt.SelectByID2("", "EDGE", -0.0341223962029176, 0.0300321434688158, 0.0460303188361877, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "EDGE", 0.0436465176948104, 0.0301916139486593, 0.0114344277122314, True, 2, Nothing, 0)

        'Create linear pattern
        swFeature = swFeatureManager.FeatureLinearPattern5(3, 0.01, 3, 0.01, False, False, "NULL", "NULL", True, False, False, False, False, False, True,
True, False, False, 0, 0, False, False)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Create Block Definition and Insert Block Instance Example (C#)

This example shows how to create a block definition and insert a block instance in a drawing.

```
//-----
// Preconditions: Open a drawing document.
//
// Postconditions:
// 1. Creates a block definition.
// 2. Inserts a first instance of the block in the upper-left corner of the
//    drawing sheet.
// 3. Inserts a second instance of the block in the drawing sheet.
//-----

using System;
using System.Collections.Generic;
using System.Text;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
namespace SketchBlocks.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            SldWorks.SldWorks swApp = new SldWorks.SldWorks();
            ModelDoc2 swModel;
            ModelDoc2 swModel2;
            DrawingDoc swDraw;
            SketchSegment[] swSkSeg = new SketchSegment[14];
            SketchPoint[] swSkPt = new SketchPoint[4];
            Note[] swSkNote = new Note[3];
            Object vSkSeg;
            Object vSkPt;
            Object vSkNote;
            SketchBlockDefinition swSketchBlockDef;
            SketchBlockInstance swBlockInst;
            SketchManager swSketchMgr;
            ModelDocExtension swModelDocExt;
            MathUtility swMathUtil;
            MathPoint swMathPoint;
            double[] nPt = new double[3];
            Object vPt;
            int nbrSelObjects;

            swModel = swApp.IActiveDoc2;
            swDraw = (DrawingDoc)swModel;

            //Interfaces needed for block APIs
            swSketchMgr = swModel.SketchManager;
            swModelDocExt = swModel.Extension;
            swMathUtil = swApp.IGetMathUtility();

            swSketchMgr.AddToDB = true;

            // Revision box
            swSkSeg[0] = (SketchSegment)swSketchMgr.CreateLine(0.008372353756316, 0.207929860362, 0, 0.05495488122133, 0.207929860362, 0);
            swSkSeg[1] = (SketchSegment)swSketchMgr.CreateLine(0.05495488122133, 0.207929860362, 0, 0.05495488122133, 0.1992788195471, 0);
            swSkSeg[2] = (SketchSegment)swSketchMgr.CreateLine(0.05495488122133, 0.1992788195471, 0, 0.008372353756316, 0.1992788195471, 0);
            swSkSeg[3] = (SketchSegment)swSketchMgr.CreateLine(0.008372353756316, 0.1992788195471, 0, 0.008372353756316, 0.207929860362, 0);
            swSkSeg[4] = (SketchSegment)swSketchMgr.CreateLine(0.023613610833382, 0.207929860362, 0, 0.023613610833382, 0.1992788195471, 0);
            swSkSeg[5] = (SketchSegment)swSketchMgr.CreateLine(0.03964919362569, 0.207929860362, 0, 0.03964919362569, 0.1992788195471, 0);

            // Clear selections; otherwise, notes are attached to line
            swModel.ClearSelection2(true);

            // Revision notes
            swSkNote[0] = (Note)swModel.InsertNote("Cell 1");
            swSkNote[1] = (Note)swModel.InsertNote("Cell 2");
            swSkNote[2] = (Note)swModel.InsertNote("Cell 3");
            PositionNote(swModel, swSkNote[0], 0.009481461553102, 0.2052680016497, 0.0);
            PositionNote(swModel, swSkNote[1], 0.025613610833382, 0.2052680016497, 0.0);
```

```

PositionNote(swModel, swSkNote[2], 0.04275469545669, 0.2052680016497, 0.0);

// Points for circles
swSkPt[0] = (SketchPoint)swSketchMgr.CreatePoint(0.02700536474232, 0.1708856599494, 0);
swSkPt[1] = (SketchPoint)swSketchMgr.CreatePoint(0.02700536474232, 0.1815330947985, 0);
swSkPt[2] = (SketchPoint)swSketchMgr.CreatePoint(0.03964919362569, 0.1815330947985, 0);
swSkPt[3] = (SketchPoint)swSketchMgr.CreatePoint(0.05029662847483, 0.1708856599494, 0);

// Circles
swSkSeg[6] = (SketchSegment)swSketchMgr.CreateCircle(swSkPt[0].X, swSkPt[0].Y, swSkPt[0].Z, 0.03050393605009, 0.169349494074, 0.0);
swSkSeg[7] = (SketchSegment)swSketchMgr.CreateCircle(swSkPt[1].X, swSkPt[1].Y, swSkPt[1].Z, 0.03305243799009, 0.183621104938, 0.0);
swSkSeg[8] = (SketchSegment)swSketchMgr.CreateCircle(swSkPt[2].X, swSkPt[2].Y, swSkPt[2].Z, 0.04426584652606, 0.182092003774, 0.0);
swSkSeg[9] = (SketchSegment)swSketchMgr.CreateCircle(swSkPt[3].X, swSkPt[3].Y, swSkPt[3].Z, 0.05496955467404, 0.164252490194, 0.0);

// Lines between circles
swSkSeg[10] = (SketchSegment)swSketchMgr.CreateLine(swSkPt[0].X, swSkPt[0].Y, swSkPt[0].Z, swSkPt[1].X, swSkPt[1].Y, swSkPt[1].Z);
swSkSeg[11] = (SketchSegment)swSketchMgr.CreateLine(swSkPt[1].X, swSkPt[1].Y, swSkPt[1].Z, swSkPt[2].X, swSkPt[2].Y, swSkPt[2].Z);
swSkSeg[12] = (SketchSegment)swSketchMgr.CreateLine(swSkPt[2].X, swSkPt[2].Y, swSkPt[2].Z, swSkPt[3].X, swSkPt[3].Y, swSkPt[3].Z);
swSkSeg[13] = (SketchSegment)swSketchMgr.CreateLine(swSkPt[3].X, swSkPt[3].Y, swSkPt[3].Z, swSkPt[0].X, swSkPt[0].Y, swSkPt[0].Z);

vSkSeg = swSkSeg;
vSkPt = swSkPt;
vSkNote = swSkNote;

// Select all sketch segments, sketch points,
// and notes for the block definition
nbrSelObjects = swModelDocExt.MultiSelect2(vSkSeg, true, null);
nbrSelObjects = swModelDocExt.MultiSelect2(vSkPt, true, null);
nbrSelObjects = swModelDocExt.MultiSelect2(vSkNote, true, null);

//Create block definition
swSketchBlockDef = swSketchMgr.MakeSketchBlockFromSelected(null);

swModel2 = (ModelDoc2)swApp.ActiveDoc;
swSketchMgr = swModel2.SketchManager;

// Define an insertion point
nPt[0] = 60.0 / 1000.0;
nPt[1] = 60.0 / 1000.0;
nPt[2] = 0.0;
vPt = nPt;
swMathPoint = (MathPoint)swMathUtil.CreatePoint(vPt);

// Insert an instance of the block definition
swBlockInst = swSketchMgr.InsertSketchBlockInstance(swSketchBlockDef, swMathPoint, 1, 0);

swSketchMgr.AddToDB = false;

// Redraw to see all changes
swModel2.GraphicsRedraw2();
}

static private void PositionNote(ModelDoc2 swModel, Note swSkNote, double X_pos, double Y_pos, double Z_pos)
{
    Annotation swAnn;
    int nRetVal;
    bool bRet;

    swAnn = (Annotation)swSkNote.GetAnnotation();

    swSkNote.Angle = 0.0;
    bRet = swSkNote.SetBalloon((int)swBalloonStyle_e.swBS_None, (int)swBalloonFit_e.swBF_Tightest);
    nRetVal = swAnn.SetLeader3((int)swLeaderStyle_e.swLBENT, (int)swLeaderSide_e.swLS_SMART, true, false, false, false);
    bRet = swAnn.SetPosition2(X_pos, Y_pos, Z_pos);

    // Redraw to see changes; however, this is expensive
    swModel.GraphicsRedraw2();
}

public SldWorks swApp;
}
}

```

Create Block Definition and Insert Block Instance Example (VBA)

This example shows how to create a block definition and insert a block instance in a drawing.

```
.....
' Preconditions: Open a drawing document.
'
' Postconditions:
' 1. Creates a block definition.
' 2. Inserts a first instance of the block in the upper-left corner of the
' drawing sheet.
' 3. Inserts a second instance of the block in the drawing sheet.
.....
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDraw As SldWorks.DrawingDoc
Dim swSkSeg(13) As SldWorks.SketchSegment
Dim swSkPt(3) As SldWorks.SketchPoint
Dim swSkNote(2) As SldWorks.Note
Dim vSkSeg As Variant
Dim vSkPt As Variant
Dim vSkNote As Variant
Dim swSketchBlockDef As SldWorks.SketchBlockDefinition
Dim swBlockInst As SldWorks.SketchBlockInstance
Dim swSketchMgr As SldWorks.SketchManager
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swMathUtil As SldWorks.MathUtility
Dim swMathPoint As SldWorks.MathPoint
Dim nPt(2) As Double
Dim vPt As Variant
Dim nbrSelObjects As Long
```

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swDraw = swModel
```

```
Set swSketchMgr = swModel.SketchManager
Set swModelDocExt = swModel.Extension
Set swMathUtil = swApp.GetMathUtility
```

```
swSketchMgr.AddToDB = True
```

```
' Revision box
Set swSkSeg(0) = swSketchMgr.CreateLine(0.008372353756316, 0.207929860362, 0#, 0.05495488122133, 0.207929860362, 0#)
Set swSkSeg(1) = swSketchMgr.CreateLine(0.05495488122133, 0.207929860362, 0#, 0.05495488122133, 0.1992788195471, 0#)
Set swSkSeg(2) = swSketchMgr.CreateLine(0.05495488122133, 0.1992788195471, 0#, 0.008372353756316, 0.1992788195471, 0#)
Set swSkSeg(3) = swSketchMgr.CreateLine(0.008372353756316, 0.1992788195471, 0#, 0.008372353756316, 0.207929860362, 0#)
Set swSkSeg(4) = swSketchMgr.CreateLine(0.023613610833382, 0.207929860362, 0#, 0.023613610833382, 0.1992788195471, 0#)
Set swSkSeg(5) = swSketchMgr.CreateLine(0.03964919362569, 0.207929860362, 0#, 0.03964919362569, 0.1992788195471, 0#)
```

```
' Clear selections; otherwise, notes are attached to line
swModel.ClearSelection2 True
```

```
' Revision notes
Set swSkNote(0) = swModel.InsertNote("Cell 1")
Set swSkNote(1) = swModel.InsertNote("Cell 2")
Set swSkNote(2) = swModel.InsertNote("Cell 3")
PositionNote swApp, swModel, swSkNote(0), 0.009481461553102, 0.2052680016497, 0#
PositionNote swApp, swModel, swSkNote(1), 0.025613610833382, 0.2052680016497, 0#
PositionNote swApp, swModel, swSkNote(2), 0.04275469545669, 0.2052680016497, 0#
```

```
' Points for circles
Set swSkPt(0) = swSketchMgr.CreatePoint(0.02700536474232, 0.1708856599494, 0#)
Set swSkPt(1) = swSketchMgr.CreatePoint(0.02700536474232, 0.1815330947985, 0#)
Set swSkPt(2) = swSketchMgr.CreatePoint(0.03964919362569, 0.1815330947985, 0#)
Set swSkPt(3) = swSketchMgr.CreatePoint(0.05029662847483, 0.1708856599494, 0#)
```

```
' Circles
```

```
Set swSkSeg(6) = swSketchMgr.CreateCircle(swSkPt(0).X, swSkPt(0).Y, swSkPt(0).Z, 0.03050393605009, 0.169349494074, 0#)
Set swSkSeg(7) = swSketchMgr.CreateCircle(swSkPt(1).X, swSkPt(1).Y, swSkPt(1).Z, 0.03305243799009, 0.183621104938, 0#)
Set swSkSeg(8) = swSketchMgr.CreateCircle(swSkPt(2).X, swSkPt(2).Y, swSkPt(2).Z, 0.04426584652606, 0.182092003774, 0#)
Set swSkSeg(9) = swSketchMgr.CreateCircle(swSkPt(3).X, swSkPt(3).Y, swSkPt(3).Z, 0.05496955467404, 0.164252490194, 0#)
```

```
' Lines between circles
```

```
Set swSkSeg(10) = swSketchMgr.CreateLine(swSkPt(0).X, swSkPt(0).Y, swSkPt(0).Z, swSkPt(1).X, swSkPt(1).Y, swSkPt(1).Z)
Set swSkSeg(11) = swSketchMgr.CreateLine(swSkPt(1).X, swSkPt(1).Y, swSkPt(1).Z, swSkPt(2).X, swSkPt(2).Y, swSkPt(2).Z)
Set swSkSeg(12) = swSketchMgr.CreateLine(swSkPt(2).X, swSkPt(2).Y, swSkPt(2).Z, swSkPt(3).X, swSkPt(3).Y, swSkPt(3).Z)
Set swSkSeg(13) = swSketchMgr.CreateLine(swSkPt(3).X, swSkPt(3).Y, swSkPt(3).Z, swSkPt(0).X, swSkPt(0).Y, swSkPt(0).Z)
```

```
vSkSeg = swSkSeg
vSkPt = swSkPt
vSkNote = swSkNote
```

```
nbrSelObjects = swModelDocExt.MultiSelect2(vSkSeg, True, Nothing)
nbrSelObjects = swModelDocExt.MultiSelect2(vSkPt, True, Nothing)
nbrSelObjects = swModelDocExt.MultiSelect2(vSkNote, True, Nothing)
```

```
Set swSketchBlockDef = swSketchMgr.MakeSketchBlockFromSelected(Nothing)
```

```
' Define an insertion point
```

```
nPt(0) = 60# / 1000#
nPt(1) = 60# / 1000#
nPt(2) = 0#
vPt = nPt
Set swMathPoint = swMathUtil.CreatePoint(vPt)
```

```
' Insert an instance of the block definition
```

```
Set swBlockInst = swSketchMgr.InsertSketchBlockInstance(swSketchBlockDef, swMathPoint, 1, 0)
```

```
swSketchMgr.AddToDB = False
```

```
' Redraw to see all changes
```

```
swModel.GraphicsRedraw2
```

```
End Sub
```

```
Sub PositionNote (swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSkNote As SldWorks.Note, X_pos As Double, Y_pos As Double, Z_pos As Double)
```

```
Dim swAnn As SldWorks.Annotation
Dim nRetVal As Long
Dim bRet As Boolean
```

```
Set swAnn = swSkNote.GetAnnotation
```

```
swSkNote.Angle = 0#
```

```
bRet = swSkNote.SetBalloon(swBS_None, swBF_Tightest)
nRetVal = swAnn.SetLeader3(swLeaderStyle_e.swBENT, swLS_SMART, True, False, False, False)
bRet = swAnn.SetPosition2(X_pos, Y_pos, Z_pos)
```

```
' Redraw to see changes
```

```
swModel.GraphicsRedraw2
```

```
End Sub
```

Create Block Definition and Insert Block Instance Example (VB.NET)

This example shows how to create a block definition and insert a block instance in a drawing.

```
'-----
' Preconditions: Open a drawing document.
'
' Postconditions:
' 1. Creates a block definition.
' 2. Inserts a first instance of the block in the upper-left corner of the
'    drawing sheet.
' 3. Inserts a second instance of the block in the drawing sheet.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()

        Dim swModel As ModelDoc2

        Dim swDraw As DrawingDoc

        Dim swSkSeg(13) As SketchSegment

        Dim swSkPt(3) As SketchPoint

        Dim swSkNote(2) As Note

        Dim vSkSeg As Object

        Dim vSkPt As Object

        Dim vSkNote As Object

        Dim swSketchBlockDef As SketchBlockDefinition

        Dim swBlockInst As SketchBlockInstance

        Dim swSketchMgr As SketchManager

        Dim swModelDocExt As ModelDocExtension

        Dim swMathUtil As MathUtility

        Dim swMathPoint As MathPoint

        Dim nPt(2) As Double

        Dim vPt As Object

        Dim nbrSelObjects As Integer

        swModel = swApp.ActiveDoc

        swDraw = swModel

        swSketchMgr = swModel.SketchManager

        swModelDocExt = swModel.Extension

        swMathUtil = swApp.GetMathUtility

        swSketchMgr.AddToDB = True

        ' Revision box

        swSkSeg(0) = swSketchMgr.CreateLine(0.008372353756316, 0.207929860362, 0.0#, 0.05495488122133, 0.207929860362, 0.0#)

        swSkSeg(1) = swSketchMgr.CreateLine(0.05495488122133, 0.207929860362, 0.0#, 0.05495488122133, 0.1992788195471, 0.0#)

        swSkSeg(2) = swSketchMgr.CreateLine(0.05495488122133, 0.1992788195471, 0.0#, 0.008372353756316, 0.1992788195471, 0.0#)

        swSkSeg(3) = swSketchMgr.CreateLine(0.008372353756316, 0.1992788195471, 0.0#, 0.008372353756316, 0.207929860362, 0.0#)

        swSkSeg(4) = swSketchMgr.CreateLine(0.023613610833382, 0.207929860362, 0.0#, 0.023613610833382, 0.1992788195471, 0.0#)

        swSkSeg(5) = swSketchMgr.CreateLine(0.03964919362569, 0.207929860362, 0.0#, 0.03964919362569, 0.1992788195471, 0.0#)
```

```

' Clear selections; otherwise, notes are attached to line
swModel.ClearSelection2(True)

' Revision notes
swSkNote(0) = swModel.InsertNote("Cell 1")
swSkNote(1) = swModel.InsertNote("Cell 2")
swSkNote(2) = swModel.InsertNote("Cell 3")
PositionNote(swApp, swModel, swSkNote(0), 0.009481461553102, 0.2052680016497, 0.0#)
PositionNote(swApp, swModel, swSkNote(1), 0.025613610833382, 0.2052680016497, 0.0#)
PositionNote(swApp, swModel, swSkNote(2), 0.04275469545669, 0.2052680016497, 0.0#)

' Points for circles
swSkPt(0) = swSketchMgr.CreatePoint(0.02700536474232, 0.1708856599494, 0.0#)
swSkPt(1) = swSketchMgr.CreatePoint(0.02700536474232, 0.1815330947985, 0.0#)
swSkPt(2) = swSketchMgr.CreatePoint(0.03964919362569, 0.1815330947985, 0.0#)
swSkPt(3) = swSketchMgr.CreatePoint(0.05029662847483, 0.1708856599494, 0.0#)

' Circles
swSkSeg(6) = swSketchMgr.CreateCircle(swSkPt(0).X, swSkPt(0).Y, swSkPt(0).Z, 0.03050393605009, 0.169349494074, 0.0#)
swSkSeg(7) = swSketchMgr.CreateCircle(swSkPt(1).X, swSkPt(1).Y, swSkPt(1).Z, 0.03305243799009, 0.183621104938, 0.0#)
swSkSeg(8) = swSketchMgr.CreateCircle(swSkPt(2).X, swSkPt(2).Y, swSkPt(2).Z, 0.04426584652606, 0.182092003774, 0.0#)
swSkSeg(9) = swSketchMgr.CreateCircle(swSkPt(3).X, swSkPt(3).Y, swSkPt(3).Z, 0.05496955467404, 0.164252490194, 0.0#)

' Lines between circles
swSkSeg(10) = swSketchMgr.CreateLine(swSkPt(0).X, swSkPt(0).Y, swSkPt(0).Z, swSkPt(1).X, swSkPt(1).Y, swSkPt(1).Z)
swSkSeg(11) = swSketchMgr.CreateLine(swSkPt(1).X, swSkPt(1).Y, swSkPt(1).Z, swSkPt(2).X, swSkPt(2).Y, swSkPt(2).Z)
swSkSeg(12) = swSketchMgr.CreateLine(swSkPt(2).X, swSkPt(2).Y, swSkPt(2).Z, swSkPt(3).X, swSkPt(3).Y, swSkPt(3).Z)
swSkSeg(13) = swSketchMgr.CreateLine(swSkPt(3).X, swSkPt(3).Y, swSkPt(3).Z, swSkPt(0).X, swSkPt(0).Y, swSkPt(0).Z)

vSkSeg = swSkSeg
vSkPt = swSkPt
vSkNote = swSkNote

nbrSelObjects = swModelDocExt.MultiSelect2(vSkSeg, True, Nothing)
nbrSelObjects = swModelDocExt.MultiSelect2(vSkPt, True, Nothing)
nbrSelObjects = swModelDocExt.MultiSelect2(vSkNote, True, Nothing)

swSketchBlockDef = swSketchMgr.MakeSketchBlockFromSelected(Nothing)

' Define an insertion point
nPt(0) = 60.0# / 1000.0#
nPt(1) = 60.0# / 1000.0#
nPt(2) = 0.0#
vPt = nPt
swMathPoint = swMathUtil.CreatePoint(vPt)

' Insert an instance of the block definition
swBlockInst = swSketchMgr.InsertSketchBlockInstance(swSketchBlockDef, swMathPoint, 1, 0)

' Turn on grid and entity snapping
swSketchMgr.AddToDB = False

```

```
' Redraw to see all changes
```

```
swModel.GraphicsRedraw2()
```

```
End Sub
```

```
Sub PositionNote(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swSkNote As Note, ByVal X_pos As Double, ByVal Y_pos
```

```
Dim swAnn As Annotation
```

```
Dim nRetVal As Integer
```

```
Dim bRet As Boolean
```

```
swAnn = swSkNote.GetAnnotation
```

```
swSkNote.Angle = 0.0#
```

```
bRet = swSkNote.SetBalloon(swBalloonStyle_e.swBS_None, swBalloonFit_e.swBF_Tightest)
```

```
nRetVal = swAnn.SetLeader3(swLeaderStyle_e.swBENT, swLeaderSide_e.swLS_SMART, True, False, False, False)
```

```
bRet = swAnn.SetPosition2(X_pos, Y_pos, Z_pos)
```

```
' Redraw to see changes
```

```
swModel.GraphicsRedraw2()
```

```
End Sub
```

```
''' <summary>
```

```
''' The SldWorks swApp variable is pre-assigned for you.
```

```
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Create Body From Selected Faces Example (C#)

This example shows how to:

- use SOLIDWORKS geometry and topology methods to construct a temporary body from selected faces.
- create a solid body feature from the temporary body and a new part containing the solid body feature.

```
//-----
// Preconditions:
// 1. Verify that the specified part document
//    template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part document and creates an
//    extruded thin feature.
// 2. Selects connecting faces on the extruded thin feature.
// 3. Knits the faces into a solid, creates a
//    a new part, and inserts the solid as an imported
//    solid body feature.
// 4. Examine the Immediate window, graphics area,
//    FeatureManager design tree, and document name
//    in the SOLIDWORKS menu bar.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CreateBodyFromFacesCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            PartDoc swPart = default(PartDoc);
            PartDoc swNewPart = default(PartDoc);
            Modeler swModeler = default(Modeler);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Face2[] swSelFace = null;
            object vFaceList = null;
            Body2 swBody = default(Body2);
            Body2 swNewBody = default(Body2);
            Feature swFeat = default(Feature);
            int nSelCount = 0;
            int i = 0;
            bool bRet = false;

            //Create part and select connecting faces
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2014\\templates\\Part.prtdot", 0, 0, 0);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.0, 0.0, 0.0, -0.062359, 0.0, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.062359, 0.0, 0.0, -0.020485, 0.066264, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.020485, 0.066264, 0.0, 0.0, 0.0, 0.0);
            swModel.ClearSelection2(true);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.FeatureExtrusionThin2(true, false, false, 0, 0, 0.03048, 0.00254, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, 0.00254, 0.00254,
            0.00254, 0, 0, false, 0.005, true, 0, 0, false);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelMgr.EnableContourSelection = false;
            swModel.ClearSelection2(true);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            bRet = swModelDocExt.SelectByID2("", "FACE", -0.0484137409629284, 0, 0.018103012393226, true, 0, null, 0);
            bRet = swModelDocExt.SelectByID2("", "FACE", -0.0396839112014504, 0.035882458904041, 0.0207108765403632, true, 0, null, 0);
            bRet = swModelDocExt.SelectByID2("", "FACE", -0.0175462018075336, 0.0567577015655729, 0.021527257415471, true, 0, null, 0);

            //Get the selected faces
            swModeler = (Modeler)swApp.GetModeler();
            nSelCount = swSelMgr.GetSelectedObjectCount();
            Array.Resize(ref swSelFace, nSelCount);
            for (i = 1; i <= nSelCount; i++)
            {
                swSelFace[i - 1] = (Face2)swSelMgr.GetSelectedObject6(i, -1);
            }
            vFaceList = (object)swSelFace;

            //Create solid body feature using selected faces
            swPart = (PartDoc)swModel;
            swBody = (Body2)swPart.CreateNewBody();
            swNewBody = (Body2)swBody.CreateBodyFromFaces(nSelCount, (vFaceList));
            if (swNewBody == null)
            {
                Debug.Print("Failed to create solid body from selected faces.");
                return;
            }
            else
            {
                Debug.Print("Solid body and new part can be created from selected faces.");
            }
            //Open new part and force creation of solid body feature
            swNewPart = (PartDoc)swApp.NewPart();
            swFeat = (Feature)swNewPart.CreateFeatureFromBody3(swNewBody, false, (int)swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck + (int)swCreateFe
atureBodyOpts_e.swCreateFeatureBodySimplify);
            if ((swFeat != null))
            {
                Debug.Print("New part with imported solid body created.");
            }
            else
            {
                Debug.Print("New part with imported solid body not created.");
            }
        }
    }
}
```

```
}  
  /// <summary>  
  ///   The Sldworks swApp variable is pre-assigned for you.  
  /// </summary>  
  public SldWorks swApp;  
}
```

Create Body From Selected Faces Example (VBA)

This example shows how to:

- use SOLIDWORKS geometry and topology methods to construct a temporary body from selected faces.
- create a solid body feature from the temporary body and a new part containing the solid body feature.

```

'-----
' Preconditions:
' 1. Verify that the specified part document
'    template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document and creates an
'    extruded thin feature.
' 2. Selects connecting faces on the extruded thin feature.
' 3. Knits the faces into a solid, creates a
'    a new part, and inserts the solid as an imported
'    solid body feature.
' 4. Examine the Immediate window, graphics area,
'    FeatureManager design tree, and document name
'    in the SOLIDWORKS menu bar.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSketchManager As SldWorks.SketchManager
    Dim swSketchSegment As SldWorks.SketchSegment
    Dim swFeatureManager As SldWorks.FeatureManager
    Dim swFeature As SldWorks.Feature
    Dim swPart As SldWorks.PartDoc
    Dim swNewPart As SldWorks.PartDoc
    Dim swModeler As SldWorks.Modeler
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelFace() As SldWorks.Face2
    Dim vFaceList As Variant
    Dim swBody As SldWorks.Body2
    Dim swNewBody As SldWorks.Body2
    Dim swFeat As SldWorks.Feature
    Dim nSelCount As Long
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")

    'Create part and select connecting faces
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    Set swSketchSegment = swSketchManager.CreateLine(0#, 0#, 0#, -0.062359, 0#, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(-0.062359, 0#, 0#, -0.020485, 0.066264, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(-0.020485, 0.066264, 0#, 0#, 0#, 0#)
    swModel.ClearSelection2 True
    Set swFeatureManager = swModel.FeatureManager
    Set swFeature = swFeatureManager.FeatureExtrusionThin2(True, False, False, 0, 0, 0.03048, 0.00254, False, False, False, False, 1.74532925199433E-02,
1.74532925199433E-02, False, False, False, False, True, 0.00254, 0.00254, 0.00254, 0, 0, False, 0.005, True, True, 0, 0, False)
    Set swSelMgr = swModel.SelectionManager
    swSelMgr.EnableContourSelection = False
    swModel.ClearSelection2 True
    Set swModelDocExt = swModel.Extension
    bRet = swModelDocExt.SelectByID("", "FACE", -4.84137409629284E-02, 0, 0.018103012393226, True, 0, Nothing, 0)
    bRet = swModelDocExt.SelectByID("", "FACE", -3.96839112014504E-02, 0.035882458904041, 2.07108765403632E-02, True, 0, Nothing, 0)
    bRet = swModelDocExt.SelectByID("", "FACE", -1.75462018075336E-02, 5.67577015655729E-02, 0.021527257415471, True, 0, Nothing, 0)

    'Get the selected faces
    Set swModeler = swApp.GetModeler
    nSelCount = swSelMgr.GetSelectedObjectCount
    ReDim swSelFace(nSelCount - 1)
    For i = 1 To nSelCount
        Set swSelFace(i - 1) = swSelMgr.GetSelectedObject6(i, -1)
    Next
    vFaceList = swSelFace

    'Create solid body feature using selected faces
    Set swPart = swModel
    Set swBody = swPart.CreateNewBody
    Set swNewBody = swBody.CreateBodyFromFaces(nSelCount, (vFaceList))
    If swNewBody Is Nothing Then
        Debug.Print "Failed to create solid body from selected faces."
        Exit Sub
    Else
        Debug.Print "Solid body and new part can be created from selected faces."
    End If

    'Open new part and force creation of solid body feature
    Set swNewPart = swApp.NewPart
    Set swFeat = swNewPart.CreateFeatureFromBody3(swNewBody, False, swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck +
swCreateFeatureBodyOpts_e.swCreateFeatureBodySimplify)
    If Not swFeat Is Nothing Then
        Debug.Print "New part with imported solid body created."
    Else

```

```
        Debug.Print "New part with imported solid body not created."  
    End If  
End Sub
```

Create Body From Selected Faces Example (VB.NET)

This example shows how to:

- use SOLIDWORKS geometry and topology methods to construct a temporary body from selected faces.
- create a solid body feature from the temporary body and a new part containing the solid body feature.

```

-----
Preconditions:
1. Verify that the specified part document
   template exists.
2. Open the Immediate window.
Postconditions:
1. Opens a new part document and creates an
   extruded thin feature.
2. Selects connecting faces on the extruded thin feature.
3. Knits the faces into a solid, creates a
   a new part, and inserts the solid as an imported
   solid body feature.
4. Examine the Immediate window, graphics area,
   FeatureManager design tree, and document name
   in the SOLIDWORKS menu bar.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim swPart As PartDoc
        Dim swNewPart As PartDoc
        Dim swModeler As Modeler
        Dim swSelMgr As SelectionMgr
        Dim swSelFace() As Face2
        Dim vFaceList As Object
        Dim swBody As Body2
        Dim swNewBody As Body2
        Dim swFeat As Feature
        Dim nSelCount As Integer
        Dim i As Integer
        Dim bRet As Boolean

        'Create part and select connecting faces
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prt", 0, 0, 0)
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)
        swSketchSegment = swSketchManager.CreateLine(0.0#, 0.0#, 0.0#, -0.062359, 0.0#, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(-0.062359, 0.0#, 0.0#, -0.020485, 0.066264, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(-0.020485, 0.066264, 0.0#, 0.0#, 0.0#, 0.0#)
        swModel.ClearSelection2(True)
        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.FeatureExtrusionThin2(True, False, False, 0, 0, 0.03048, 0.00254, False, False, False, False, 0.0174532925199433, 0.0174532925199433, False, False, True, 0.00254, 0.00254, 0.00254, 0, 0, False, 0.005, True, True, 0, 0, False)
        swSelMgr = swModel.SelectionManager
        swSelMgr.EnableContourSelection = False
        swModel.ClearSelection2(True)
        swModelDocExt = swModel.Extension
        bRet = swModelDocExt.SelectByID("", "FACE", -0.0484137409629284, 0, 0.018103012393226, True, 0, Nothing, 0)
        bRet = swModelDocExt.SelectByID("", "FACE", -0.0396839112014504, 0.035882458904041, 0.0207108765403632, True, 0, Nothing, 0)
        bRet = swModelDocExt.SelectByID("", "FACE", -0.0175462018075336, 0.0567577015655729, 0.021527257415471, True, 0, Nothing, 0)

        'Get the selected faces
        swModeler = swApp.GetModeler
        nSelCount = swSelMgr.GetSelectedObjectCount
        ReDim swSelFace(nSelCount - 1)
        For i = 1 To nSelCount
            swSelFace(i - 1) = swSelMgr.GetSelectedObject6(i, -1)
        Next
        vFaceList = swSelFace

        'Create solid body feature using selected faces
        swPart = swModel
        swBody = swPart.CreateNewBody
        swNewBody = swBody.CreateBodyFromFaces(nSelCount, (vFaceList))
        If swNewBody Is Nothing Then
            Debug.Print("Failed to create solid body from selected faces.")
            Exit Sub
        Else
            Debug.Print("Solid body and new part can be created from selected faces.")
        End If

        'Open new part and force creation of solid body feature
        swNewPart = swApp.NewPart
        swFeat = swNewPart.CreateFeatureFromBody3(swNewBody, False, swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck + swCreateFeatureBodyOpts_e.swCreateFeatureBodySimplify)
        If Not swFeat Is Nothing Then
            Debug.Print("New part with imported solid body created.")
        Else
            Debug.Print("New part with imported solid body not created.")
        End If

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create Body Using Trimmed Surfaces Example (C#)

The basic outline for creating a body using trimmed surfaces is as follows:

- 1. Create a new temporary body in a part using IPartDoc::CreateNewBody.
- 2. Create trimmed surfaces to create a body (for example, create six square surfaces that intersect at the edges to form a cube):
 - a. Create a planar surface using IBody2::CreatePlanarSurface(RootPoint, Normal).
 - b. Add a trimming loop to the planar surface using ISurface::AddTrimmingLoop2(Numcurves, _Order, _Dimen, _Periodic, _NumKnots, _NumCtrlPoints, _Knots, _CtrlPointDbis, _UVRange)
 - c. The arguments for Surface::AddTrimmingLoop2 and their values for a square trimming loop are:

Argument	Description
NumCurves	Number of spline curves that make up the trimming loop (Long 4)
Order	Orders for the spline curves ({2, 2, 2, 2} Array of Longs. Second-order linear curves)
Dimen	Dimension of the control points for the spline curves ({2, 2, 2, 2} Array of Longs)
Periodic	Periodicity of the spline curves ({0, 0, 0, 0} Array of Longs)
NumKnots	Number of knots of the spline curves ({4, 4, 4, 4} Array of Longs)
NumCtrlPoints	Number of control points for the spline curves ({2, 2, 2, 2} Array of Longs)
Knots	Describes the locations of the knots on the spline curves ({0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1} Array of Doubles. Each spline curve contains 4 knots)
CtrlPointDbis	Coordinates of control points for the square trimming loop ({0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0} Array of Doubles. (Two for each spline curve of the square trimming loop * Dimen) = 16 coordinates for the square trimming loop)
UVRange	Min and max for the U and V values ({0, 1, 0, 1} Array of Doubles)

- d. Create a trimmed surface on the body based on the trimming loop that was just created.
- 3. Sew the surfaces together into a new body using IBody2::CreateBodyFromSurfaces.

This example shows how to create trimmed surfaces and use them to create bodies.

```
//-----
// Preconditions: Verify that the specified
// part document template exists.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates a temporary body.
// 3. Creates six trimmed surfaces. For each:
//   a. Creates a planar surface.
//   b. Adds a trimming loop to the planar surface.
//   c. Creates a trimmed surface on the temporary body based
//       on the trimming loop.
// 4. Sews the six trimmed surfaces together to form a new cube body.
// 5. Creates a trimmed surface that is offset from the back of the cube.
// 6. Examine the FeatureManager design tree and
//     graphics area.
//-----
```

```
using System.Windows.Forms;
using System;
using System.Runtime.InteropServices;
using SolidWorks.Interop.sldworks;
```

```
namespace AddTrimmingLoop2_CSharp
{
    public partial class SolidWorksMacro
```

```

{
    private ModelDoc2 Model;
    private PartDoc Part;
    private double[] RootPoint = new double[3];
    private double[] Normal = new double[3];
    private Body2 TempBody;
    private bool isGood;
    public void Main()
    {

        swApp.UserControl = true;

        // Create a new blank document
        Model = (ModelDoc2)swApp.NewDocument(@"C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2020\templates\part.prtdot", 0, 0, 0);
        Part = (PartDoc)Model;

        // Create a new temporary body in the part
        TempBody = (Body2)Part.CreateNewBody();
        if (TempBody == null)
        {
            MessageBox.Show("Could not create the new body.");
            return;
        }

        // Create trimmed surfaces for a cube with one meter per side

        // LEFT
        RootPoint[0] = 0; // X
        RootPoint[1] = 0; // Y
        RootPoint[2] = 0; // Z
        Normal[0] = 1; // X
        Normal[1] = 0; // Y
        Normal[2] = 0; // Z
        isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, true);

        // RIGHT
        RootPoint[0] = 1;
        RootPoint[1] = 0;
        RootPoint[2] = 0;
        Normal[0] = 1;
        Normal[1] = 0;
        Normal[2] = 0;
        isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, true);

        // BOTTOM
        RootPoint[0] = 0;
        RootPoint[1] = 0;
        RootPoint[2] = -1;
        Normal[0] = 0;
        Normal[1] = 1;
        Normal[2] = 0;
        isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, true);

        // TOP
        RootPoint[0] = 0;
        RootPoint[1] = 1;
        RootPoint[2] = -1;
        Normal[0] = 0;
        Normal[1] = 1;
        Normal[2] = 0;
        isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, true);
    }
}

```

```

// FRONT
RootPoint[0] = 0;
RootPoint[1] = 0;
RootPoint[2] = 0;
Normal[0] = 0;
Normal[1] = 0;
Normal[2] = 1;
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, true);

// BACK
RootPoint[0] = 0;
RootPoint[1] = 0;
RootPoint[2] = -1;
Normal[0] = 0;
Normal[1] = 0;
Normal[2] = 1;
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, true);

// Create the body from the trimmed surfaces just created
TempBody.CreateBodyFromSurfaces();

// Create an offset surface from the back
RootPoint[0] = 0;
RootPoint[1] = 0;
RootPoint[2] = -2;
Normal[0] = 0;
Normal[1] = 0;
Normal[2] = 1;
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, false);

Model.ViewZoomtofit2();
}

```

```

// CreateSquareSurface creates a square trimmed surface
private bool CreateSquareSurface(PartDoc Part, Body2 SurfaceBody, object RootPoint, object Normal, bool
IsPartOfTempBody)
{
    bool isGood;
    // Temporary surface
    Surface TmpSurf;
    // Arguments that define the trimming loop
    int NumCurves;
    int[] Order = new int[4];
    int[] Dimen = new int[4];
    int[] Periodic = new int[4];
    int[] NumKnots = new int[4];
    int[] NumCtrlPoints = new int[4];
    double[] Knots = new double[16];
    double[] CtrlPointDbls = new double[16];
    double[] UVRange = new double[4];
    // Initially this function has no problems,
    // set this value to false if errors encountered
    //CreateSquareSurface = true;
    // Create a new planar surface based at RootPoint with the Normal vector Normal
    TmpSurf = (Surface)SurfaceBody.CreatePlanarSurface(RootPoint, Normal);
    if (TmpSurf == null)
    {
        //CreateSquareSurface = false;
        return false;
    }
    // Set the arguments to define a square trimming loop
    // There are four curves (four sides)

```

```

    NumCurves = 4;
    // Orders of the spline curves
    Order[0] = 2;
    Order[1] = 2;
    Order[2] = 2;
    Order[3] = 2;
    // There are two dimensions
    Dimen[0] = 2;
    Dimen[1] = 2;
    Dimen[2] = 2;
    Dimen[3] = 2;
    // No periodics
    Periodic[0] = 0;
    Periodic[1] = 0;
    Periodic[2] = 0;
    Periodic[3] = 0;
    // There are four knots (corners)
    NumKnots[0] = 4;
    NumKnots[1] = 4;
    NumKnots[2] = 4;
    NumKnots[3] = 4;
    // A square has two control points
    NumCtrlPoints[0] = 2;
    NumCtrlPoints[1] = 2;
    NumCtrlPoints[2] = 2;
    NumCtrlPoints[3] = 2;
    // The locations of the knots
    Knots[0] = 0;
    Knots[1] = 0;
    Knots[2] = 1;
    Knots[3] = 1;
    Knots[4] = 0;
    Knots[5] = 0;
    Knots[6] = 1;
    Knots[7] = 1;
    Knots[8] = 0;
    Knots[9] = 0;
    Knots[10] = 1;
    Knots[11] = 1;
    Knots[12] = 0;
    Knots[13] = 0;
    Knots[14] = 1;
    Knots[15] = 1;
    // Set the actual trimming corners
    CtrlPointDbls[0] = 0; CtrlPointDbls[1] = 0;
    CtrlPointDbls[2] = 1; CtrlPointDbls[3] = 0;
    CtrlPointDbls[4] = 1; CtrlPointDbls[5] = 0;
    CtrlPointDbls[6] = 1; CtrlPointDbls[7] = 1;
    CtrlPointDbls[8] = 1; CtrlPointDbls[9] = 1;
    CtrlPointDbls[10] = 0; CtrlPointDbls[11] = 1;
    CtrlPointDbls[12] = 0; CtrlPointDbls[13] = 1;
    CtrlPointDbls[14] = 0; CtrlPointDbls[15] = 0;
    // The possible range for the UV values
    UVRange[0] = 0;
    UVRange[1] = 1;
    UVRange[2] = 0;
    UVRange[3] = 1;
    // Create the trimming loop on the surface
    isGood = TmpSurf.AddTrimmingLoop2(NumCurves, Order, Dimen, Periodic, NumKnots, NumCtrlPoints, Knots, CtrlPointDbls,
    UVRange);

    if (isGood == false)
    {
        //CreateSquareSurface = false;

```

```

        return false;
    }

    // Create the trimmed surface on the body
    // based on the trimming loop just created
    isGood = SurfaceBody.CreateTrimmedSurface();
    if (isGood == false)
    {
        //CreateSquareSurface = false;
        return false;
    }

    if (IsPartOfTempBody)
    {
        //do nothing
    }
    else
    {
        SurfaceBody.CreateBodyFromSurfaces();

        Marshal.ReleaseComObject(TmpSurf);

        GC.WaitForPendingFinalizers();
        GC.Collect();

        GC.WaitForPendingFinalizers();
        GC.Collect();

        return true;
    }

    // The SldWorks swApp variable is pre-assigned for you.
    public SldWorks swApp;

}
}

```

Create Body Using Trimmed Surfaces Example (VBA)

The basic outline for creating a body using trimmed surfaces is as follows:

- 1. Create a new temporary body in a part using IPartDoc::CreateNewBody.
- 2. Create trimmed surfaces to create a body (for example, create six square surfaces that intersect at the edges to form a cube):
 - a. Create a planar surface using IBody2::CreatePlanarSurface(RootPoint, Normal).
 - b. Add a trimming loop to the planar surface using ISurface::AddTrimmingLoop2(Numcurves, _Order, _Dimen, _Periodic, _NumKnots, _NumCtrlPoints, _Knots, _CtrlPointDbis, _UVRange)
 - c. The arguments for Surface::AddTrimmingLoop2 and their values for a square trimming loop are:

Argument	Description
NumCurves	Number of spline curves that make up the trimming loop (Long 4)
Order	Orders for the spline curves ({2, 2, 2, 2} Array of Longs. Second-order linear curves)
Dimen	Dimension of the control points for the spline curves ({2, 2, 2, 2} Array of Longs)
Periodic	Periodicity of the spline curves ({0, 0, 0, 0} Array of Longs)
NumKnots	Number of knots of the spline curves ({4, 4, 4, 4} Array of Longs)
NumCtrlPoints	Number of control points for the spline curves ({2, 2, 2, 2} Array of Longs)
Knots	Describes the locations of the knots on the spline curves ({0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1} Array of Doubles. Each spline curve contains 4 knots)
CtrlPointDbis	Coordinates of control points for the square trimming loop ({0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0} Array of Doubles. (Two for each spline curve of the square trimming loop * Dimen) = 16 coordinates for the square trimming loop)
UVRange	Min and max for the U and V values ({0, 1, 0, 1} Array of Doubles)

- d. Create a trimmed surface on the body based on the trimming loop that was just created.
- 3. Sew the surfaces together into a new body using IBody2::CreateBodyFromSurfaces.

This example shows how to create trimmed surfaces and use them to create bodies.

```
' -----
' Preconditions: Verify that the specified
' part document template exists.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a temporary body.
' 3. Creates six trimmed surfaces. For each:
'   a. Creates a planar surface.
'   b. Adds a trimming loop to the planar surface.
'   c. Creates a trimmed surface on the temporary body based
'       on the trimming loop.
' 4. Sews the six trimmed surfaces together to form a new cube body.
' 5. Creates a trimmed surface that is offset from the back of the cube.
' 6. Examine the FeatureManager design tree and
'    graphics area.
' -----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Model As SldWorks.ModelDoc2
Dim Part As SldWorks.PartDoc
Dim RootPoint(2) As Double
Dim Normal(2) As Double
Dim TempBody As SldWorks.Body2
Dim isGood As Boolean

Sub main()

'Get the SOLIDWORKS application
Set swApp = CreateObject("SldWorks.Application")
swApp.UserControl = True

'Create a new blank document
Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2020\templates\part.prtdot", 0, 0, 0)
Set Part = Model

'Create a new temporary body in the part
Set TempBody = Part.CreateNewBody

If TempBody Is Nothing Then
    MsgBox "Could not create the new body."
    Exit Sub
End If

'Create the trimmed surfaces for a cube with one meter length sides

'LEFT
```

```

RootPoint(0) = 0 'X
RootPoint(1) = 0 'Y
RootPoint(2) = 0 'Z
Normal(0) = 1 'X
Normal(1) = 0 'Y
Normal(2) = 0 'Z
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, True)

'RIGHT
RootPoint(0) = 1
RootPoint(1) = 0
RootPoint(2) = 0
Normal(0) = 1
Normal(1) = 0
Normal(2) = 0
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, True)

'BOTTOM
RootPoint(0) = 0
RootPoint(1) = 0
RootPoint(2) = -1
Normal(0) = 0
Normal(1) = 1
Normal(2) = 0
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, True)

'TOP
RootPoint(0) = 0
RootPoint(1) = 1
RootPoint(2) = -1
Normal(0) = 0
Normal(1) = 1
Normal(2) = 0
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, True)

'FRONT
RootPoint(0) = 0
RootPoint(1) = 0
RootPoint(2) = 0
Normal(0) = 0
Normal(1) = 0
Normal(2) = 1
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, True)

'BACK
RootPoint(0) = 0
RootPoint(1) = 0
RootPoint(2) = -1
Normal(0) = 0
Normal(1) = 0
Normal(2) = 1
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, True)

'Create the body using the trimmed surfaces just created
TempBody.CreateBodyFromSurfaces

'Create an offset surface from the back of the cube body
RootPoint(0) = 0
RootPoint(1) = 0
RootPoint(2) = -2
Normal(0) = 0
Normal(1) = 0
Normal(2) = 1
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, False)

Model.ViewZoomtofit2

'Clean up the memory
Set swApp = Nothing
Set Model = Nothing
Set Part = Nothing

End Sub

'CreateSquareSurface creates a square trimmed surface

Private Function CreateSquareSurface(Part As PartDoc, SurfaceBody As Body2, RootPoint As Variant, Normal As Variant, IsPartOfTempBody As Boolean) As Boolean
    Dim isGood As Boolean

    'Temporary surface
    Dim TmpSurf As SldWorks.Surface

    'Arguments that define the trimming loop
    Dim NumCurves As Long
    Dim Order(3) As Long
    Dim Dimen(3) As Long
    Dim Periodic(3) As Long
    Dim NumKnots(3) As Long
    Dim NumCtrlPoints(3) As Long
    Dim Knots(15) As Double
    Dim CtrlPointDbIs(15) As Double
    Dim UVRRange(3) As Double

    'Initially this function has no problems,
    'set this value to false if errors encountered
    CreateSquareSurface = True

    'Create a new planar surface based at RootPoint with the normal vector Normal
    Set TmpSurf = SurfaceBody.CreatePlanarSurface(RootPoint, Normal)
    If TmpSurf Is Nothing Then
        CreateSquareSurface = False
    
```

```

Exit Function
End If

'Set the arguments to define a square trimming loop

'There are four spline curves (four sides) in the square
NumCurves = 4

'Order of each of the four sides of the square
Order(0) = 2
Order(1) = 2
Order(2) = 2
Order(3) = 2

'Dimensions for the control points of each side of the square
Dimen(0) = 2
Dimen(1) = 2
Dimen(2) = 2
Dimen(3) = 2

'Non-periodic sides of the square
Periodic(0) = 0
Periodic(1) = 0
Periodic(2) = 0
Periodic(3) = 0

'Number of knots in each side of the square
NumKnots(0) = 4
NumKnots(1) = 4
NumKnots(2) = 4
NumKnots(3) = 4

'Number of control points in each side of the square
NumCtrlPoints(0) = 2
NumCtrlPoints(1) = 2
NumCtrlPoints(2) = 2
NumCtrlPoints(3) = 2

'Sixteen knots, four for each side of the square
Knots(0) = 0
Knots(1) = 0
Knots(2) = 1
Knots(3) = 1
Knots(4) = 0
Knots(5) = 0
Knots(6) = 1
Knots(7) = 1
Knots(8) = 0
Knots(9) = 0
Knots(10) = 1
Knots(11) = 1
Knots(12) = 0
Knots(13) = 0
Knots(14) = 1
Knots(15) = 1

'16 control point coordinates for the square
CtrlPointDbls(0) = 0: CtrlPointDbls(1) = 0
CtrlPointDbls(2) = 1: CtrlPointDbls(3) = 0
CtrlPointDbls(4) = 1: CtrlPointDbls(5) = 0
CtrlPointDbls(6) = 1: CtrlPointDbls(7) = 1
CtrlPointDbls(8) = 1: CtrlPointDbls(9) = 1
CtrlPointDbls(10) = 0: CtrlPointDbls(11) = 1
CtrlPointDbls(12) = 0: CtrlPointDbls(13) = 1
CtrlPointDbls(14) = 0: CtrlPointDbls(15) = 0

'UV ranges
UVRange(0) = 0
UVRange(1) = 1
UVRange(2) = 0
UVRange(3) = 1

'Create the trimming loop on the surface
isGood = TmpSurf.AddTrimmingLoop2(NumCurves, Order, Dimen, Periodic, NumKnots, NumCtrlPoints, Knots, CtrlPointDbls, UVRange)
If isGood = False Then
    CreateSquareSurface = False
    Exit Function
End If

'Create the trimmed surface on the body
'based on the trimming loop just created
isGood = SurfaceBody.CreateTrimmedSurface
If isGood = False Then
    CreateSquareSurface = False
    Exit Function
End If

If IsPartOfTempBody Then
    'If this surface is to be used in
    'a temporary body, then you must generate it
Else
    SurfaceBody.CreateBodyFromSurfaces
End If

End Function

```

Create Body Using Trimmed Surfaces Example (VB.NET)

The basic outline for creating a body using trimmed surfaces is as follows:

- 1. Create a new temporary body in a part using IPartDoc::CreateNewBody.
- 2. Create trimmed surfaces to create a body (for example, create six square surfaces that intersect at the edges to form a cube):
 - a. Create a planar surface using IBody2::CreatePlanarSurface(RootPoint, Normal).
 - b. Add a trimming loop to the planar surface using ISurface::AddTrimmingLoop2(Numcurves, _Order, _Dimen, _Periodic, _NumKnots, _NumCtrlPoints, _Knots, _CtrlPointDbis, _UVRange)
 - c. The arguments for Surface::AddTrimmingLoop2 and their values for a square trimming loop are:

Argument	Description
NumCurves	Number of spline curves that make up the trimming loop (Long 4)
Order	Orders for the spline curves ({2, 2, 2, 2} Array of Longs. Second-order linear curves)
Dimen	Dimension of the control points for the spline curves ({2, 2, 2, 2} Array of Longs)
Periodic	Periodicity of the spline curves ({0, 0, 0, 0} Array of Longs)
NumKnots	Number of knots of the spline curves ({4, 4, 4, 4} Array of Longs)
NumCtrlPoints	Number of control points for the spline curves ({2, 2, 2, 2} Array of Longs)
Knots	Describes the locations of the knots on the spline curves ({0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1} Array of Doubles. Each spline curve contains 4 knots)
CtrlPointDbis	Coordinates of control points for the square trimming loop ({0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0} Array of Doubles. (Two for each spline curve of the square trimming loop * Dimen) = 16 coordinates for the square trimming loop)
UVRange	Min and max for the U and V values ({0, 1, 0, 1} Array of Doubles)

- d. Create a trimmed surface on the body based on the trimming loop that was just created.
- 3. Sew the surfaces together into a new body using IBody2::CreateBodyFromSurfaces.

This example shows how to create trimmed surfaces and use them to create bodies.

```
' -----
' Preconditions: Verify that the specified
' part document template exists.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a temporary body.
' 3. Creates six trimmed surfaces. For each:
'   a. Creates a planar surface.
'   b. Adds a trimming loop to the planar surface.
'   c. Creates a trimmed surface on the temporary body based
'      on the trimming loop.
' 4. Sews the six trimmed surfaces together to form a new cube body.
' 5. Creates a trimmed surface that is offset from the back of the cube.
' 6. Examine the FeatureManager design tree and
'    graphics area.
' -----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

Partial Class SolidWorksMacro

```
Dim Model As ModelDoc2
Dim Part As PartDoc
Dim RootPoint(2) As Double
```

```

Dim Normal(2) As Double
Dim TempBody As Body2
Dim isGood As Boolean
Sub main()

    swApp.UserControl = True

    'Create a new blank document
    Model = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2020\templates\part.prtdot", 0, 0, 0)
    Part = Model

    'Create a new temporary body in the part
    TempBody = Part.CreateNewBody
    If TempBody Is Nothing Then
        MsgBox("Could not create the new body.")
        Exit Sub
    End If

    'Create trimmed surfaces for a cube with one meter per side

    'LEFT
    RootPoint(0) = 0 'X
    RootPoint(1) = 0 'Y
    RootPoint(2) = 0 'Z
    Normal(0) = 1 'X
    Normal(1) = 0 'Y
    Normal(2) = 0 'Z
    isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, True)

    'RIGHT
    RootPoint(0) = 1
    RootPoint(1) = 0
    RootPoint(2) = 0
    Normal(0) = 1
    Normal(1) = 0
    Normal(2) = 0
    isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, True)

    'BOTTOM
    RootPoint(0) = 0
    RootPoint(1) = 0
    RootPoint(2) = -1
    Normal(0) = 0
    Normal(1) = 1
    Normal(2) = 0
    isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, True)

    'TOP
    RootPoint(0) = 0
    RootPoint(1) = 1
    RootPoint(2) = -1
    Normal(0) = 0
    Normal(1) = 1
    Normal(2) = 0
    isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, True)

    'FRONT
    RootPoint(0) = 0
    RootPoint(1) = 0
    RootPoint(2) = 0
    Normal(0) = 0
    Normal(1) = 0

```

```
Normal(2) = 1
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, True)
```

```
'BACK
RootPoint(0) = 0
RootPoint(1) = 0
RootPoint(2) = -1
Normal(0) = 0
Normal(1) = 0
Normal(2) = 1
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, True)
```

```
'Create the body from the trimmed surfaces just created
TempBody.CreateBodyFromSurfaces()
```

```
'Create an offset surface from the back
RootPoint(0) = 0
RootPoint(1) = 0
RootPoint(2) = -2
Normal(0) = 0
Normal(1) = 0
Normal(2) = 1
isGood = CreateSquareSurface(Part, TempBody, RootPoint, Normal, False)
```

```
Model.ViewZoomtofit2
```

```
End Sub
```

```
'CreateSquareSurface creates a square trimmed surface
```

```
Private Function CreateSquareSurface(Part As PartDoc, SurfaceBody As Body2, RootPoint As Object, Normal As Object,
IsPartOfTempBody As Boolean) As Boolean
```

```
Dim isGood As Boolean
'Temporary surface
Dim TmpSurf As Surface
'Arguments that define the trimming loop
Dim NumCurves As Integer
Dim Order(3) As Integer
Dim Dimen(3) As Integer
Dim Periodic(3) As Integer
Dim NumKnots(3) As Integer
Dim NumCtrlPoints(3) As Integer
Dim Knots(15) As Double
Dim CtrlPointDbls(15) As Double
Dim UVRange(3) As Double
'Initially this function has no problems,
'set this value to false if errors encountered
CreateSquareSurface = True
'Create a new planar surface based at RootPoint with the Normal vector Normal
TmpSurf = SurfaceBody.CreatePlanarSurface(RootPoint, Normal)
If TmpSurf Is Nothing Then
    CreateSquareSurface = False
    Exit Function
End If
'Set the arguments to define a square trimming loop
'There are four curves (four sides)
NumCurves = 4
'Orders of the spline curves
Order(0) = 2
Order(1) = 2
Order(2) = 2
Order(3) = 2
'There are two dimensions
```

```

Dimen(0) = 2
Dimen(1) = 2
Dimen(2) = 2
Dimen(3) = 2
'No periodics
Periodic(0) = 0
Periodic(1) = 0
Periodic(2) = 0
Periodic(3) = 0
'There are four knots (corners)
NumKnots(0) = 4
NumKnots(1) = 4
NumKnots(2) = 4
NumKnots(3) = 4
'A square has two control points
NumCtrlPoints(0) = 2
NumCtrlPoints(1) = 2
NumCtrlPoints(2) = 2
NumCtrlPoints(3) = 2
'The locations of the knots
Knots(0) = 0
Knots(1) = 0
Knots(2) = 1
Knots(3) = 1
Knots(4) = 0
Knots(5) = 0
Knots(6) = 1
Knots(7) = 1
Knots(8) = 0
Knots(9) = 0
Knots(10) = 1
Knots(11) = 1
Knots(12) = 0
Knots(13) = 0
Knots(14) = 1
Knots(15) = 1
'Set the actual trimming corners
CtrlPointDbls(0) = 0 : CtrlPointDbls(1) = 0
CtrlPointDbls(2) = 1 : CtrlPointDbls(3) = 0
CtrlPointDbls(4) = 1 : CtrlPointDbls(5) = 0
CtrlPointDbls(6) = 1 : CtrlPointDbls(7) = 1
CtrlPointDbls(8) = 1 : CtrlPointDbls(9) = 1
CtrlPointDbls(10) = 0 : CtrlPointDbls(11) = 1
CtrlPointDbls(12) = 0 : CtrlPointDbls(13) = 1
CtrlPointDbls(14) = 0 : CtrlPointDbls(15) = 0
'The possible range for the UV values
UVRange(0) = 0
UVRange(1) = 1
UVRange(2) = 0
UVRange(3) = 1
'Create the trimming loop on the surface
isGood = TmpSurf.AddTrimmingLoop2(NumCurves, Order, Dimen, Periodic, NumKnots, NumCtrlPoints, Knots, CtrlPointDbls,
UVRange)
If isGood = False Then
    CreateSquareSurface = False
    Exit Function
End If

'Create the trimmed surface on the body
'based on the trimming loop just created
isGood = SurfaceBody.CreateTrimmedSurface
If isGood = False Then
    CreateSquareSurface = False

```

```

        Exit Function
    End If

    If IsPartOfTempBody Then
        'If this surface is to be used in
        'a temporary body, then you must generate it
    Else
        SurfaceBody.CreateBodyFromSurfaces()
    End If

    Marshal.ReleaseComObject(TmpSurf)

    GC.WaitForPendingFinalizers()
    GC.Collect()

    GC.WaitForPendingFinalizers()
    GC.Collect()
End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Create Bounding Box Example (C#)

This example shows how to create a bounding box.

```
//-----
// Preconditions: Open an Immediate window and ensure that the specified model to open exists.
//
// Postconditions:
// 1. Creates Bounding Box in the FeatureManager design tree.
// 2. Modifies Bounding Box.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateBoundingBox_CSharp
{
    partial class SolidWorksMacro
    {
        private BoundingBoxFeatureData swGPData;
        private BoundingBoxFeatureData featdata;
        private ModelDoc2 Part;
        private Face2 ent;
        private Feature feat;
        private FeatureManager featmgr;
        private bool boolstatus;
        private int longstatus = 0;
        private int longwarnings = 0;

        public void Main()
        {
            Part = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2024\\samples\\tutorial\\api\\block20.sldprt");
            swApp.ActivateDoc2("block20.sldprt", false, longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;

            featmgr = Part.FeatureManager;

            boolstatus = Part.Extension.SelectByRay(0.0108472195143463, 0.0396239999998329, -0.00101823136031953, -0.400036026779312);

            ent = (Face2)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            swGPData = (BoundingBoxFeatureData)featmgr.CreateDefinition((int)swFeatureNameID_e.swFmBoundingBox);
            swGPData.ReferenceFaceOrPlane = 2;
            swGPData.PlanarEntity = ent;

            feat = featmgr.CreateFeature(swGPData);
            Part.ClearSelection2(true);

            featdata = (BoundingBoxFeatureData)feat.GetDefinition();
            featdata.AccessSelections(Part, null);
            boolstatus = Part.Extension.SelectByRay(-0.0103569711794194, 0.0188454182651299, 0.049345602378537, -0.400036026779312);
            ent = (Face2)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            featdata.PlanarEntity = ent;
            featdata.IncludeSurfaces = true;
            featdata.IncludeHiddenBodies = true;
            feat.ModifyDefinition(featdata, Part, null);

            Debug.Print("Diameter(m) of sphere enclosing the bounding box: " + Part.Extension.GetSphericalBoxDiameter());
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Creating Bounding Box Example (VBA)

This example shows how to create a bounding box.

```

'-----
' Preconditions: Open an Immediate window and ensure that the specified model to open exists.
'
' Postconditions:
' 1. Creates Bounding Box in the FeatureManager design tree.
' 2. Modifies Bounding Box.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

Dim swApp As SldWorks.SldWorks
Dim swGPData As SldWorks.BoundingBoxFeatureData
Dim featdata As SldWorks.BoundingBoxFeatureData
Dim Part As SldWorks.ModelDoc2
Dim ent As SldWorks.Face2
Dim feat As SldWorks.Feature
Dim featmgr As SldWorks.FeatureManager
Dim boolstatus As Boolean
Dim longstatus As Long, longwarnings As Long
Option Explicit
Sub main()
    Set swApp = Application.SldWorks

    Set Part = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2024\samples\tutorial\api\block20.sldprt", 1, 0, "",
longstatus, longwarnings)
    swApp.ActivateDoc2 "block20.sldprt", False, longstatus
    Set Part = swApp.ActiveDoc

    Set featmgr = Part.FeatureManager

    boolstatus = Part.Extension.SelectByRay(1.08472195143463E-02, 3.96239999998329E-02, -1.01823136031953E-03, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 1.1224765174324E-03, 2, False, 0, 0)

    Set ent = Part.SelectionManager.GetSelectedObject6(1, -1)
    Set swGPData = featmgr.CreateDefinition(swFmBoundingBox)
    swGPData.ReferenceFaceOrPlane = 2
    swGPData.PlanarEntity = ent

    Set feat = featmgr.CreateFeature(swGPData)
    Part.ClearSelection2 True

    Set featdata = feat.GetDefinition()
    featdata.AccessSelections Part, Nothing
    boolstatus = Part.Extension.SelectByRay(-1.03569711794194E-02, 1.88454182651299E-02, 0.049345602378537, -0.400036026779312,
-0.515038074910024, -0.758094294050284, 1.1224765174324E-03, 2, False, 0, 0)
    Set ent = Part.SelectionManager.GetSelectedObject6(1, -1)
    featdata.PlanarEntity = ent
    featdata.IncludeSurfaces = True
    featdata.IncludeHiddenBodies = True
    feat.ModifyDefinition featdata, Part, Nothing

    Debug.Print "Diameter(m) of sphere enclosing the bounding box: " & Part.Extension.GetSphericalBoxDiameter()

End Sub

```

Create Broken-Out Section Example (C#)

This example shows how to create a broken-out section in a drawing view.

```
//-----
// Preconditions:
// 1. Open a drawing.
// 2. Select Drawing View1.
//
// Postconditions: A broken-out section is created in Drawing View1
// using the specified closed spline.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CreateBreakOutSection_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        object pointArray;
        double[] points = new double[12];
        SketchSegment skSegment;
        SelectData selectData = null;
        DrawingDoc dDoc;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;

            points[0] = -0.0544316967839374;
            points[1] = 0.0413619530906299;
            points[2] = 0;
            points[3] = 0.0530556603589196;
            points[4] = 0.0413619530906299;
            points[5] = 0;
            points[6] = 0.00783232107320536;
            points[7] = 0.00720299635749822;
            points[8] = 0;
            points[9] = -0.0544316967839374;
            points[10] = 0.0413619530906299;
            points[11] = 0;

            pointArray = points;

            skSegment = Part.SketchManager.CreateSpline((pointArray));
            skSegment.Select4(true, selectData);

            dDoc = (DrawingDoc)Part;
            dDoc.CreateBreakOutSection(0.00254);

            Part.ClearSelection2(true);
        }

        public SldWorks swApp;
    }
}
```

Create Broken-Out Section Example (VBA)

This example shows how to create a broken-out section in a drawing view.

```
'-----  
' Preconditions:  
' 1. Open a drawing.  
' 2. Select Drawing View1.  
'  
' Postconditions: A broken-out section is created in Drawing View1  
' using the specified closed spline.  
'-----
```

```
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim pointArray As Variant  
Dim points() As Double  
Dim skSegment As SldWorks.SketchSegment  
Dim selectData As SldWorks.selectData  
  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
  
    ReDim points(0 To 11) As Double  
    points(0) = -5.44316967839374E-02  
    points(1) = 4.13619530906299E-02  
    points(2) = 0  
    points(3) = 5.30556603589196E-02  
    points(4) = 4.13619530906299E-02  
    points(5) = 0  
    points(6) = 7.83232107320536E-03  
    points(7) = 7.20299635749822E-03  
    points(8) = 0  
    points(9) = -5.44316967839374E-02  
    points(10) = 4.13619530906299E-02  
    points(11) = 0  
  
    pointArray = points  
  
    Set skSegment = Part.SketchManager.CreateSpline((pointArray))  
    skSegment.Select4 True, selectData  
  
    Part.CreateBreakOutSection 0.00254  
  
    Part.ClearSelection2 True  
  
End Sub
```

Create Broken-Out Section Example (VB.NET)

This example shows how to create a broken-out section in a drawing view.

```
'-----  
' Preconditions:  
' 1. Open a drawing.  
' 2. Select Drawing View1.  
'-----  
' Postconditions: A broken-out section is created in Drawing View1  
' using the specified closed spline.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2  
    Dim pointArray As Object  
    Dim points(11) As Double  
    Dim skSegment As SketchSegment  
    Dim selectData As SelectData
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc
```

```
        points(0) = -0.0544316967839374  
        points(1) = 0.0413619530906299  
        points(2) = 0  
        points(3) = 0.0530556603589196  
        points(4) = 0.0413619530906299  
        points(5) = 0  
        points(6) = 0.00783232107320536  
        points(7) = 0.00720299635749822  
        points(8) = 0  
        points(9) = -0.0544316967839374  
        points(10) = 0.0413619530906299  
        points(11) = 0
```

```
        pointArray = points
```

```
        skSegment = Part.SketchManager.CreateSpline((pointArray))  
        skSegment.Select4(True, selectData)
```

```
        Part.CreateBreakOutSection(0.00254)
```

```
        Part.ClearSelection2(True)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Create Break View Example (C#)

This example shows how to create and remove a broken view.

```
//-----
// Preconditions:
// 1. Verify that the specified file to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing and selects Drawing View1.
// 2. Examine the drawing, then press F5.
// 3. Inserts break lines in Drawing View1.
// 4. Examine the drawing, then press F5.
// 5. Modifies the positions of the break lines and breaks the view.
// 6. Examine the drawing, then press F5.
// 7. Removes the break from Drawing View1.
// 8. Examine the drawing and the Immediate window.
//
// NOTE: Because this drawing document is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace BreakViewDrawingDocCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            DrawingDoc swDrawingDoc;
            ModelDocExtension swModelDocExt;
            SelectionMgr swSelectionManager;
            SelectData swSelectData;
            View swView;
            BreakLine swBreakLine;
            string fileName;
            bool status;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\box.slddrw";
            swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDrawingDoc = (DrawingDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Activate and select the view to break

            status = swDrawingDoc.ActivateView("Drawing View1");

            status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);

            swSelectionManager = (SelectionMgr)swModel.SelectionManager;
            swSelectData = (SelectData)swSelectionManager.CreateSelectData();

            swView = (View)swSelectionManager.GetSelectedObject6(1, -1);

            System.Diagnostics.Debugger.Break();

            // Examine the drawing; press F5

            // Insert the break lines

            swBreakLine = (BreakLine)swView.InsertBreak(0, -0.0291950859897372, 0.0198236302285804, 1);

            System.Diagnostics.Debugger.Break();

            // Break lines inserted; press F5

            // Reset position of break lines

            status = swBreakLine.SetPosition(-0.03, 0.05);
            swModel.EditRebuild3

            Debug.Print("Break line: ");

            Debug.Print("
Selected: " + swBreakLine.Select(true, null));

            Debug.Print(" Style: " + swBreakLine.Style);

            Debug.Print(" Orientation: " + swBreakLine.Orientation);

            Debug.Print(" Position: " + swBreakLine.GetPosition(0));

            swDrawingDoc.BreakView();

            System.Diagnostics.
```

```
Debugger.Break();
    // Positions of the break lines are modified, and view is broken
    // Press F5

    status = swModelDocExt.SelectByID(
"Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
    swDrawingDoc.UnBreakView();

    // Break is removed

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}
```

Create Break View Example (VBA)

This example shows how to create and remove a broken view.

```
'-----
' Preconditions:
' 1. Verify that the specified file to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing and selects Drawing View1.
' 2. Examine the drawing, then press F5.
' 3. Inserts break lines in Drawing View1.
' 4. Examine the drawing, then press F5.
' 5. Modifies the positions of the break lines and breaks the view.
' 6. Examine the drawing, then press F5.
' 7. Removes the break from Drawing View1.
' 8. Examine the drawing and the Immediate window.
'
' NOTE: Because this drawing document is used elsewhere,
' do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim swSheet As SldWorks.Sheet
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionManager As SldWorks.SelectionMgr
Dim swSelectData As SldWorks.SelectData
Dim swView As SldWorks.View
Dim swBreakLine As SldWorks.BreakLine
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks

    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.slddrw"

    Set swModel = swApp.OpenDoc6(fileName, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)

    Set swDrawingDoc = swModel

    Set swModelDocExt = swModel.Extension

    ' Activate and select the view to break
    status = swDrawingDoc.ActivateView("Drawing View1")
    status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionManager = swModel.SelectionManager
    Set swSelectData = swSelectionManager.CreateSelectData
    Set swView = swSelectionManager.GetSelectedObject6(1, -1)

    Stop
    ' Examine the drawing; press F5

    Set swBreakLine = swView.InsertBreak(2, -2.91950859897372E-02, 1.98236302285804E-02, 2)

    Stop
    ' Break lines inserted; press F5

    status = swBreakLine.SetPosition(-0.03, 0.05)
    swModel.EditRebuild3
    Debug.Print ("Break line: ")
    Debug.Print (" Selected: " & swBreakLine.Select(True, Nothing))
    Debug.Print (" Style: " & swBreakLine.Style)
    Debug.Print (" Orientation: " & swBreakLine.Layer)
    Debug.Print (" Position: " & swBreakLine.GetPosition(0))

    swDrawingDoc.BreakView

    Stop
    ' Positions of the break lines are modified, and the view is broken
    ' Press F5

    status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    swDrawingDoc.UnBreakView

    ' Break is removed

End Sub
```

Create Break View Example (VB.NET)

This example shows how to create and remove a broken view.

```

' -----
' Preconditions:
' 1. Verify that the specified file to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing and selects Drawing View1.
' 2. Examine the drawing, then press F5.
' 3. Inserts break lines in Drawing View1.
' 4. Examine the drawing, then press F5.
' 5. Modifies the positions of the break lines and breaks the view.
' 6. Examine the drawing, then press F5.
' 7. Removes the break from Drawing View1.
' 8. Examine the drawing and the Immediate window.
'
' NOTE: Because this drawing document is used elsewhere,
' do not save changes.
' -----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

Public Sub Main()

```

    Dim swModel As ModelDoc2
    Dim swDrawingDoc As DrawingDoc
    Dim swModelDocExt As ModelDocExtension
    Dim swSelectionManager As SelectionMgr
    Dim swSelectData As SelectData
    Dim swView As View
    Dim swBreakLine As BreakLine
    Dim fileName As String
    Dim status As Boolean
    Dim errors As Integer
    Dim warnings As Integer

```

```

    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.slddrw"
    swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warn
    swDrawingDoc = swModel
    swModelDocExt = swModel.Extension

```

```

    ' Activate and select the view to break
    status = swDrawingDoc.ActivateView("Drawing View1")
    status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    swSelectionManager = swModel.SelectionManager
    swSelectData = swSelectionManager.CreateSelectData
    swView = swSelectionManager.GetSelectedObject6(1, -1)

```

Stop

' Examine the drawing; press F5

' Insert the break lines

```
swBreakLine = swView.InsertBreak(0, -0.0291950859897372, 0.0198236302285804, 1)
```

Stop

' Break lines inserted; press F5

' Reset position of break lines

```
status = swBreakLine.SetPosition(-0.03, 0.05)
```

swModel.EditRebuild3

```
Debug.Print("Break line: ")
```

```
Debug.Print(" Selected: " & swBreakLine.Select(True, Nothing))
```

```
Debug.Print(" Style: " & swBreakLine.Style)
```

```
Debug.Print(" Orientation: " & swBreakLine.Orientation)
```

```
Debug.Print(" Position: " & swBreakLine.GetPosition(0))
```

```
swDrawingDoc.BreakView()
```

Stop

' Positions of the break lines are modified, and the view is broken

' Press F5

' Remove the break lines

```
status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
```

```
swDrawingDoc.UnBreakView()
```

' Break is removed

End Sub

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Create a Cam-Follower Mate Example (C#)

This example shows how to create a cam-follower mate.

```
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\MechanicalMates\Cam-Follower.sldasm.
// 2. Delete CamMateCoincident1 from the Mates folder in the FeatureManager
//    design tree.
//
// Postconditions:
// Inspect the Mates folder in the FeatureManager design tree.
//
// NOTE: Because the model is used elsewhere, do not save changes.
// -----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CamFollowerMate_CSharp
{
    partial class SolidWorksMacro
    {

        private ModelDoc2 Part;
        private AssemblyDoc assy;
        private bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            assy = (AssemblyDoc)Part;

            boolstatus = Part.Extension.SelectByRay(-0.00229201081452857, 0.0761655216381314, 0.00981125242168446, -0.577381545199981, -0.577287712085548, -0.577381545199979, 0.00229201081452857);
            boolstatus = Part.Extension.SelectByRay(0, 0.0762, 0, -0.577381545199981, -0.577287712085548, -0.577381545199979, 0.00229201081452857);

            // Create CamFollowerMateFeatureData
            CamFollowerMateFeatureData MateData = (CamFollowerMateFeatureData)assy.CreateMateData(9);

            // Set the Entities To Mate
            object FirstEntityToMate = null;
            object SecondEntityToMate = null;
            FirstEntityToMate = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            MateData.EntitiesToMate[0] = FirstEntityToMate;
            SecondEntityToMate = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(2, -1);
            MateData.EntitiesToMate[1] = SecondEntityToMate;

            // Set the Mate Alignment
            MateData.MateAlignment = 2;

            // Create the mate
            Feature MateFeature = (Feature)assy.CreateMate(MateData);

            Part.EditRebuild3();
        }
        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Circular Pattern Example (C#)

This example shows how to create a circular-pattern feature using selected direction axis, pattern seed features, and variable spacing between pattern instances.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\varyinstance.sldprt
//
// Postconditions: Creates a circular-pattern feature.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateCircularPatternCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureMgr = default(FeatureManager);
            bool boolstatus = false;
            int status = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc3("varyinstance.sldprt", false, (int)swRebuildOnActivation_e.swUserDecision, ref status);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            boolstatus = swModelDocExt.SelectByID2("Cut-
Extrude1", "BODYFEATURE", 0.00843730075439453, 0.00364341890551145, -0.0354416044676498, false, 4, null, 0);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to select a seed feature"); }

            boolstatus = swModelDocExt.SelectByID2("", "EDGE", 0.00628473027779819, -0.168045059787516, -0.0496550391792034, true, 1, null, 0);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to select an edge for direction 1"); }

            boolstatus = swModelDocExt.SelectByID2("Fillet1", "BODYFEATURE", 0.000782948437176856, 0.00455320522434022, -0.0350770617062892, true, 4, null
, 0);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to select a seed feature"); }

            swFeatureMgr = (FeatureManager)swModel.FeatureManager;

            boolstatus = swFeatureMgr.InsertVaryInstanceIncrement("D1@Sketch2@varyInstance.SLDPRT", 4, 1, 0, 0.003);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to add an increment value to dimension D1@Sketch2@varyInstance.SLDPRT"); }

            boolstatus = swFeatureMgr.InsertVaryInstanceIncrement("D1@Cut-Extrude1@varyInstance.SLDPRT", 4, 1, 0, -0.001);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to add an increment value to dimension D1@Cut-Extrude1@varyInstance.SLDPRT"); }

            boolstatus = swFeatureMgr.InsertVaryInstanceIncrement("D1@Fillet1@varyInstance.SLDPRT", 4, 1, 0, 0.0001);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to add an increment value to dimension D1@Fillet1@varyInstance.SLDPRT"); }

            boolstatus = swFeatureMgr.InsertVaryInstanceIncrement("Space Increment", 4, 2, 0, 0.0349065850398866);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to add an increment value to direction 1 spacing"); }

            boolstatus = swFeatureMgr.InsertVaryInstanceOverride("D1@Sketch2@varyInstance.SLDPRT", 4, 1, -1, 5, -1, 0.05);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to override value of dimension D1@Sketch2@varyInstance.SLDPRT at instance (5, 0)"); }

            boolstatus = swFeatureMgr.InsertVaryInstanceOverride("D1@Sketch2@varyInstance.SLDPRT", 4, 1, -1, 3, -1, 0.06);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to override value of dimension D1@Sketch2@varyInstance.SLDPRT at instance (3, 0)"); }

            boolstatus = swFeatureMgr.InsertVaryInstanceOverride("D1@Cut-Extrude1@varyInstance.SLDPRT", 4, 1, -1, 5, -1, 0.005);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to override value of dimension D1@Cut-Extrude1@varyInstance.SLDPRT at instance (5, 0)"); }

            boolstatus = swFeatureMgr.InsertVaryInstanceOverride("D1@Fillet1@varyInstance.SLDPRT", 4, 1, -1, 5, -1, 0.006);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to override value of dimension D1@Fillet1@varyInstance.SLDPRT at instance (5, 0)"); }

            boolstatus = swFeatureMgr.InsertVaryInstanceOverride("D1@Fillet1@varyInstance.SLDPRT", 4, 1, -1, 3, -1, 0.004);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to override value of dimension D1@Fillet1@varyInstance.SLDPRT at instance (3, 0)"); }

            boolstatus = swFeatureMgr.InsertVaryInstanceOverride("Space Increment", 4, 2, 0, 5, -1, 1.30899693899575);
            if (boolstatus == false) { ErrorMsg(swApp, "Failed to override value of direction 1 spacing increment at instance (3, 0)"); }

            Feature myFeature = default(Feature);
            myFeature = (Feature)swFeatureMgr.FeatureCircularPattern4(6, 0.174532925199434, true, "NULL", false, false, true);
            if (myFeature == null) { ErrorMsg(swApp, "Failed to create a vary instance circular pattern"); }

        }

        public string ErrorMsg(SldWorks SwApp, string Message)
        {
            SwApp.SendMsgToUser2(Message, 0, 0);
            SwApp.RecordLine("***** WARNING - General");
            SwApp.RecordLine("***** " + Message);
            SwApp.RecordLine("");
            return "";
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Create Circular Pattern Example (VBA)

This example shows how to create a circular-pattern feature using selected direction axis, pattern seed features, and variable spacing between pattern instances.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\api\varyinstance.sldprt.
'
' Postconditions: Creates a circular-pattern feature.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Dim SwApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim boolstatus As Boolean
Dim longstatus As Long
Option Explicit
Sub main()
    Set SwApp = Application.SldWorks

    SwApp.ActivateDoc3 "varyInstance", False, swUserDecision, longstatus
    Set Part = SwApp.ActiveDoc

    boolstatus = Part.Extension.SelectByID2("Cut-Extrude1", "BODYFEATURE", 8.43730075439453E-03, 3.64341890551145E-03,
-3.54416044676498E-02, False, 4, Nothing, 0)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to select a seed feature": GoTo LastLine

    boolstatus = Part.Extension.SelectByID2("", "EDGE", 6.28473027779819E-03, -0.168045059787516, -4.96550391792034E-02, True, 1,
Nothing, 0)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to select an edge for direction 1": GoTo LastLine

    boolstatus = Part.Extension.SelectByID2("Fillet1", "BODYFEATURE", 7.82948437176856E-04, 4.55320522434022E-03,
-3.50770617062892E-02, True, 4, Nothing, 0)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to select a seed feature": GoTo LastLine

    boolstatus = Part.FeatureManager.InsertVaryInstanceIncrement("D1@Sketch2@varyInstance.SLDPRT", 4, 1, 0, 0.003)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to add an increment value to dimension D1@Sketch2@varyInstance.SLDPRT": GoTo
LastLine

    boolstatus = Part.FeatureManager.InsertVaryInstanceIncrement("D1@Cut-Extrude1@varyInstance.SLDPRT", 4, 1, 0, -0.001)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to add an increment value to dimension D1@Cut-Extrude1@varyInstance.SLDPRT":
GoTo LastLine

    boolstatus = Part.FeatureManager.InsertVaryInstanceIncrement("D1@Fillet1@varyInstance.SLDPRT", 4, 1, 0, 0.0001)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to add an increment value to dimension D1@Fillet1@varyInstance.SLDPRT": GoTo
LastLine

    boolstatus = Part.FeatureManager.InsertVaryInstanceIncrement("Space Increment", 4, 2, 0, 3.49065850398866E-02)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to add an increment value to direction 1 spacing": GoTo LastLine

    boolstatus = Part.FeatureManager.InsertVaryInstanceOverride("D1@Sketch2@varyInstance.SLDPRT", 4, 1, -1, 5, -1, 0.05)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to override value of dimension D1@Sketch2@varyInstance.SLDPRT at instance
(5, 0)": GoTo LastLine

    boolstatus = Part.FeatureManager.InsertVaryInstanceOverride("D1@Sketch2@varyInstance.SLDPRT", 4, 1, -1, 3, -1, 0.06)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to override value of dimension D1@Sketch2@varyInstance.SLDPRT at instance
(3, 0)": GoTo LastLine

    boolstatus = Part.FeatureManager.InsertVaryInstanceOverride("D1@Cut-Extrude1@varyInstance.SLDPRT", 4, 1, -1, 5, -1, 0.005)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to override value of dimension D1@Cut-Extrude1@varyInstance.SLDPRT at
instance (5, 0)": GoTo LastLine

    boolstatus = Part.FeatureManager.InsertVaryInstanceOverride("D1@Fillet1@varyInstance.SLDPRT", 4, 1, -1, 5, -1, 0.006)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to override value of dimension D1@Fillet1@varyInstance.SLDPRT at instance
(5, 0)": GoTo LastLine

    boolstatus = Part.FeatureManager.InsertVaryInstanceOverride("D1@Fillet1@varyInstance.SLDPRT", 4, 1, -1, 3, -1, 0.004)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to override value of dimension D1@Fillet1@varyInstance.SLDPRT at instance
(3, 0)": GoTo LastLine

    boolstatus = Part.FeatureManager.InsertVaryInstanceOverride("Space Increment", 4, 2, 0, 5, -1, 1.30899693899575)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to override value of direction 1 spacing increment at instance (3, 0)": GoTo
LastLine

    Dim myFeature As SldWorks.Feature
    Set myFeature = Part.FeatureManager.FeatureCircularPattern4(6, 0.174532925199434, True, "NULL", False, False, True)
    If myFeature Is Nothing Then ErrorMsg SwApp, "Failed to create a vary instance circular pattern": GoTo LastLine

LastLine:

```

```
End Sub

Function ErrorMsg(SwApp As Object, Message As String)
    SwApp.SendMsgToUser2 Message, 0, 0
    SwApp.RecordLine "**** WARNING - General"
    SwApp.RecordLine "**** " & Message
    SwApp.RecordLine ""
End Function
```

Create Circular Pattern Example (VB.NET)

This example shows how to create a circular-pattern feature using selected direction axis, pattern seed features, and variable spacing between pattern instances.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\api\varyinstance.sldprt
'
' Postconditions: Creates a circular-pattern feature.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureMgr As FeatureManager
        Dim boolstatus As Boolean
        Dim status As Integer

        swModel = swApp.ActivateDoc3("varyInstance.sldprt", False, swRebuildOnActivation_e.swUserDecision, status)
        swModelDocExt = swModel.Extension

        boolstatus = swModelDocExt.SelectByID("Cut-
Extrude1", "BODYFEATURE", 0.00843730075439453, 0.00364341890551145, -0.0354416044676498, False, 4, Nothing, 0)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to select a seed feature") : GoTo LastLine

        boolstatus = swModelDocExt.SelectByID("", "EDGE", 0.00628473027779819, -0.168045059787516, -0.0496550391792034, True, 1, Nothing, 0)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to select an edge for direction 1") : GoTo LastLine

        boolstatus = swModelDocExt.SelectByID("Fillet1", "BODYFEATURE", 0.000782948437176856, 0.00455320522434022, -0.0350770617062892, True, 4, Nothing,
0)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to select a seed feature") : GoTo LastLine

        swFeatureMgr = swModel.FeatureManager

        boolstatus = swFeatureMgr.InsertVaryInstanceIncrement("D1@Sketch2@varyInstance.SLDPRT", 4, 1, 0, 0.003)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to add an increment value to dimension D1@Sketch2@varyInstance.SLDPRT") : GoTo LastLine

        boolstatus = swFeatureMgr.InsertVaryInstanceIncrement("D1@Cut-Extrude1@varyInstance.SLDPRT", 4, 1, 0, -0.001)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to add an increment value to dimension D1@Cut-Extrude1@varyInstance.SLDPRT") : GoTo LastLine

        boolstatus = swFeatureMgr.InsertVaryInstanceIncrement("D1@Fillet1@varyInstance.SLDPRT", 4, 1, 0, 0.0001)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to add an increment value to dimension D1@Fillet1@varyInstance.SLDPRT") : GoTo LastLine

        boolstatus = swFeatureMgr.InsertVaryInstanceIncrement("Space Increment", 4, 2, 0, 0.0349065850398866)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to add an increment value to direction 1 spacing") : GoTo LastLine

        boolstatus = swFeatureMgr.InsertVaryInstanceOverride("D1@Sketch2@varyInstance.SLDPRT", 4, 1, -1, 5, -1, 0.05)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to override value of dimension D1@Sketch2@varyInstance.SLDPRT at instance (5, 0)") : GoTo LastL
ine

        boolstatus = swFeatureMgr.InsertVaryInstanceOverride("D1@Sketch2@varyInstance.SLDPRT", 4, 1, -1, 3, -1, 0.06)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to override value of dimension D1@Sketch2@varyInstance.SLDPRT at instance (3, 0)") : GoTo LastL
ine

        boolstatus = swFeatureMgr.InsertVaryInstanceOverride("D1@Cut-Extrude1@varyInstance.SLDPRT", 4, 1, -1, 5, -1, 0.005)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to override value of dimension D1@Cut-
Extrude1@varyInstance.SLDPRT at instance (5, 0)") : GoTo LastLine

        boolstatus = swFeatureMgr.InsertVaryInstanceOverride("D1@Fillet1@varyInstance.SLDPRT", 4, 1, -1, 5, -1, 0.006)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to override value of dimension D1@Fillet1@varyInstance.SLDPRT at instance (5, 0)") : GoTo LastL
ine

        boolstatus = swFeatureMgr.InsertVaryInstanceOverride("D1@Fillet1@varyInstance.SLDPRT", 4, 1, -1, 3, -1, 0.004)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to override value of dimension D1@Fillet1@varyInstance.SLDPRT at instance (3, 0)") : GoTo LastL
ine

        boolstatus = swFeatureMgr.InsertVaryInstanceOverride("Space Increment", 4, 2, 0, 5, -1, 1.30899693899575)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to override value of direction 1 spacing increment at instance (3, 0)") : GoTo LastLine

        Dim myFeature As Feature
        myFeature = swFeatureMgr.FeatureCircularPattern4(6, 0.174532925199434, True, "NULL", False, False, True)
        If myFeature Is Nothing Then ErrorMsg(swApp, "Failed to create a vary instance circular pattern") : GoTo LastLine

LastLine:

    End Sub

    Function ErrorMsg(ByVal SwApp As Object, ByVal Message As String) As String
        SwApp.SendMsgToUser2(Message, 0, 0)
        SwApp.RecordLine("***** WARNING - General")
        SwApp.RecordLine("***** " & Message)
        SwApp.RecordLine("")
        Return ""
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Create Circular Pattern of Subassembly Example (C#)

This example shows how to create a circular pattern of a subassembly.

```
//-----
// Preconditions: Verify that the assembly exists.
//
// Postconditions:
// 1. Opens the assembly.
// 2. Selects an edge for Direction 1.
// 3. Selects a subassembly to pattern.
// 4. Creates a circular pattern.
// 5. Examine the FeatureManager design tree and
//    graphics area.
//
// NOTE: Because the assembly is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\distance linkage.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            status = swModelDocExt.SelectByID("", "EDGE", 0.22639417933982, -0.194822643434378, 0.102086175644843, false, 2, null, 0);
            status = swModelDocExt.SelectByID("mount base-1@distance linkage", "COMPONENT", 0, 0, 0, true, 1, null, 0);
            swFeature = (Feature)swFeatureManager.FeatureCircularPattern5(3, 2.0943951023932, false, "NULL", false, true, false, false, false, false, 0,
            0, "NULL", false);
            swModel.ClearSelection2(true);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Circular Pattern of Subassembly Example (VBA)

This example shows how to create a circular pattern of a subassembly.

Preconditions: Verify that the assembly exists.

Postconditions:

1. Opens the assembly.
2. Selects an edge for Direction 1.
3. Selects a subassembly to pattern.
4. Creates a circular pattern.
5. Examine the FeatureManager design tree and graphics area.

NOTE: Because the assembly is used elsewhere, do not save changes.

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\distance linkage.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swFeatureManager = swModel.FeatureManager
    status = swModelDocExt.SelectByID("", "EDGE", 0.22639417933982, -0.194822643434378, 0.102086175644843, False, 2, Nothing, 0)
    status = swModelDocExt.SelectByID("mount base-1@distance linkage", "COMPONENT", 0, 0, 0, True, 1, Nothing, 0)
    Set swFeature = swFeatureManager.FeatureCircularPattern5(3, 2.0943951023932, False, "NULL", False, True, False, False, False, False, 0, 0, "NULL",
False)
    swModel.ClearSelection2 True

End Sub
```

Create Circular Pattern of Subassembly Example (VB.NET)

This example shows how to create a circular pattern of a subassembly.

 Preconditions: Verify that the assembly exists.

Postconditions:
 1. Opens the assembly.
 2. Selects an edge for Direction 1.
 3. Selects a subassembly to pattern.
 4. Creates a circular pattern.
 5. Examine the FeatureManager design tree and graphics area.

NOTE: Because the assembly is used elsewhere, do not save changes.

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\distance linkage.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swFeatureManager = swModel.FeatureManager
        status = swModelDocExt.SelectByID("", "EDGE", 0.22639417933982, -0.194822643434378, 0.102086175644843, False, 2, Nothing, 0)
        status = swModelDocExt.SelectByID("mount base-1@distance linkage", "COMPONENT", 0, 0, 0, True, 1, Nothing, 0)
        swFeature = swFeatureManager.FeatureCircularPatterns5(3, 2.0943951023932, False, "NULL", False, True, False, False, False, False, 0, 0, "NULL",
False)
    )

        swModel.ClearSelection2(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks
```

```
End Class
```